

```
In [1]: import pandas as pd
import yfinance as yf
from datetime import datetime
import matplotlib.pyplot as plt
import numpy as np
import seaborn as sb
import statsmodels.api as sm
from sklearn import metrics
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import RandomizedSearchCV
from sklearn.svm import SVR
from sklearn.model_selection import GridSearchCV
from sklearn.tree import DecisionTreeRegressor
from sklearn.neural_network import MLPRegressor
from sklearn.datasets import load_iris
from sklearn.neighbors import KNeighborsRegressor
```

```
In [2]: def get_stock_data(ticker, start_date, end_date):
    # Download stock data
    data = yf.download(ticker, start=start_date, end=end_date)
    return data

# Set the ticker symbols, start date, and end date
nse_ticker = "^NSEI" # NSE
bovespa_ticker = "^BVSP" # BOVESPA
start_date = "2018-01-01"
end_date = datetime.today().strftime('%Y-%m-%d')

# Get the stock data for both NSE and BOVESPA
nse_data = get_stock_data(nse_ticker, start_date, end_date)
bovespa_data = get_stock_data(bovespa_ticker, start_date, end_date)

# Drop NA values
nse_data.dropna(inplace=True)
bovespa_data.dropna(inplace=True)

# Combine the dataframes on the Date index
nse_data = pd.concat([nse_data, bovespa_data], axis=0)

# Display the combined data
print(nse_data)
```

```
[*****100%*****] 1 of 1 completed
[*****100%*****] 1 of 1 completed
```

	Open	High	Low	Close \
Date				
2018-01-02	10477.549805	10495.200195	10404.650391	10442.200195
2018-01-03	10482.650391	10503.599609	10429.549805	10443.200195
2018-01-04	10469.400391	10513.000000	10441.450195	10504.799805
2018-01-05	10534.250000	10566.099609	10520.099609	10558.849609
2018-01-08	10591.700195	10631.200195	10588.549805	10623.599609
...	...	...	...	...
2023-09-18	118759.000000	119486.000000	118123.000000	118288.000000
2023-09-19	118293.000000	118458.000000	117628.000000	117846.000000
2023-09-20	117864.000000	119616.000000	117847.000000	118695.000000
2023-09-21	118695.000000	118695.000000	116013.000000	116145.000000
2023-09-22	0.000000	0.000000	0.000000	116008.640625

	Adj Close	Volume
Date		
2018-01-02	10442.200195	153400
2018-01-03	10443.200195	167300
2018-01-04	10504.799805	174900
2018-01-05	10558.849609	180900
2018-01-08	10623.599609	169000
...	...	...
2023-09-18	118288.000000	11273100
2023-09-19	117846.000000	11453800
2023-09-20	118695.000000	9238800
2023-09-21	116145.000000	12685800
2023-09-22	116008.640625	0

[2833 rows x 6 columns]

```
In [3]: # Calculate Stock High minus Low price (H-L)
nse_data['H-L'] = nse_data['High'] - nse_data['Low']

# Calculate Stock Open minus Close price (O-C)
nse_data['O-C'] = nse_data['Open'] - nse_data['Close']
```

```
# Calculate Stock price's seven days' moving average (7 DAYS MA)
nse_data['7 DAYS MA'] = nse_data['Close'].rolling(window=7).mean()

# Calculate Stock price's fourteen days' moving average (14 DAYS MA)
nse_data['14 DAYS MA'] = nse_data['Close'].rolling(window=14).mean()

# Calculate Stock price's twenty one days' moving average (21 DAYS MA)
nse_data['21 DAYS MA'] = nse_data['Close'].rolling(window=21).mean()

# Calculate Stock price's standard deviation for the past seven days (7 DAYS STD DEV)
nse_data['7 DAYS STD DEV'] = nse_data['Close'].rolling(window=7).std()

# Print the data
nse_data.dropna(inplace=True)
```

```
In [4]: # Calculate descriptive statistics
statistics = nse_data.describe()

# Print the statistics
statistics
```

```
Out[4]:
```

	Open	High	Low	Close	Adj Close	Volume	H-L	O-C
count	2813.000000	2813.000000	2813.000000	2813.000000	2813.000000	2.813000e+03	2813.000000	2813.000000
mean	58846.677395	59378.937640	58305.338455	58895.770247	58895.770247	4.564711e+06	1073.599185	-49.092853
std	45552.234866	45990.064257	45126.054305	45567.439485	45567.439485	5.132731e+06	1222.048846	2451.640962
min	0.000000	0.000000	0.000000	7610.250000	7610.250000	0.000000e+00	0.000000	-116008.640625
25%	13473.500000	13548.900391	13399.299805	13513.849609	13513.849609	3.183000e+05	139.900391	-187.000000
50%	71178.000000	72247.000000	69779.000000	71394.000000	71394.000000	9.977000e+05	690.900391	0.750000
75%	104745.000000	105652.000000	103851.000000	104781.000000	104781.000000	8.891000e+06	1760.000000	180.250000
max	130776.000000	131190.000000	129526.000000	130776.000000	130776.000000	2.602930e+07	16615.000000	12520.000000

```
In [5]: # Calculate correlation matrix
correlation_matrix = nse_data.corr()

# Print the correlation matrix
correlation_matrix
```

```
Out[5]:
```

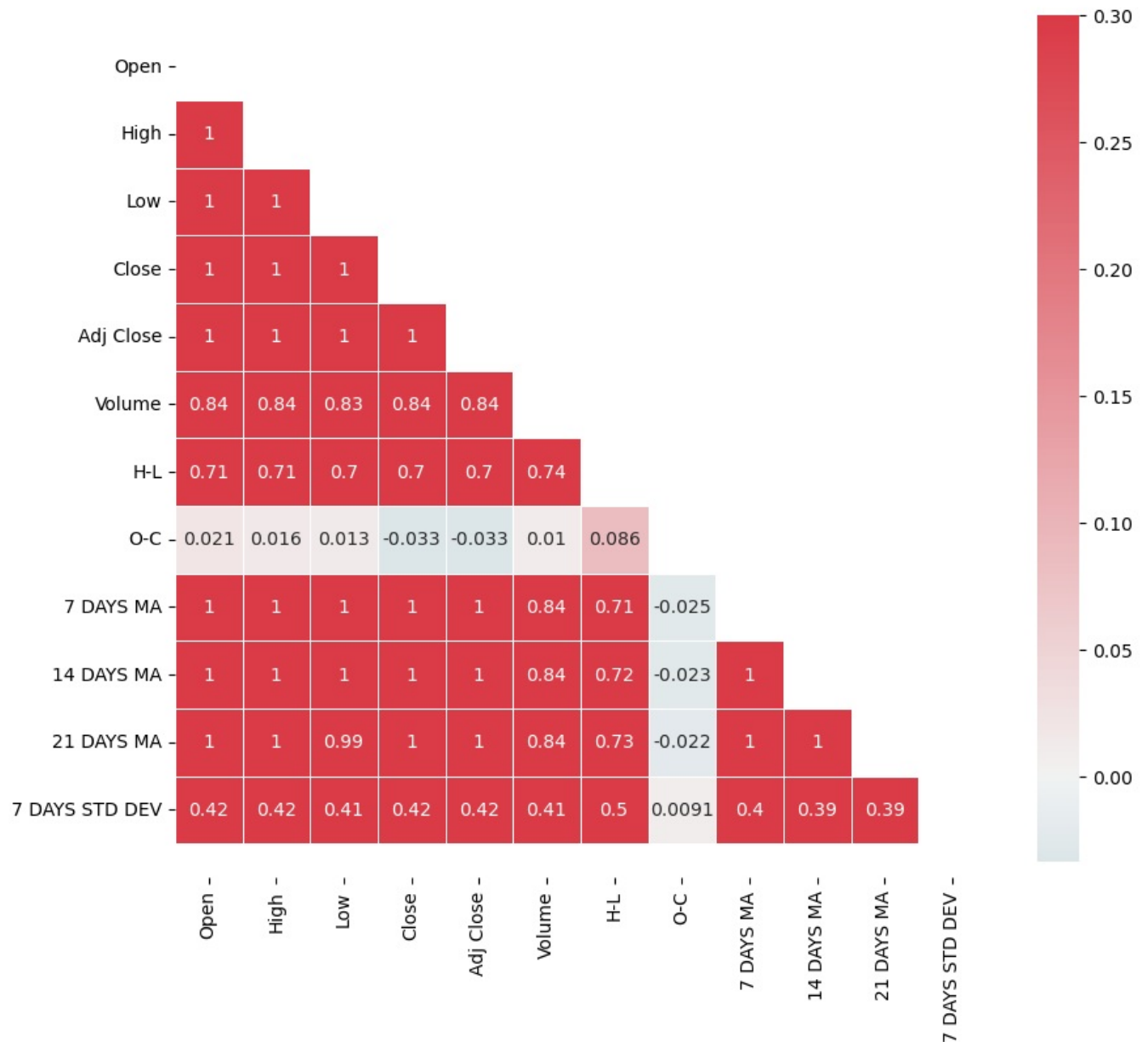
	Open	High	Low	Close	Adj Close	Volume	H-L	O-C	7 DAYS MA	14 DAYS MA	21 DAYS MA	7 DAY STD DEV
Open	1.000000	0.999882	0.999809	0.998552	0.998552	0.836508	0.709627	0.020707	0.997970	0.996655	0.995152	0.415781
High	0.999882	1.000000	0.999820	0.998714	0.998714	0.838505	0.713662	0.015517	0.997977	0.996744	0.995310	0.417860
Low	0.999809	0.999820	1.000000	0.998749	0.998749	0.834444	0.700246	0.013494	0.997735	0.996248	0.994625	0.412403
Close	0.998552	0.998714	0.998749	1.000000	1.000000	0.835674	0.704782	-0.033102	0.998957	0.997559	0.996012	0.415152
Adj Close	0.998552	0.998714	0.998749	1.000000	1.000000	0.835674	0.704782	-0.033102	0.998957	0.997559	0.996012	0.415152
Volume	0.836508	0.838505	0.834444	0.835674	0.835674	1.000000	0.742795	0.010311	0.837809	0.840299	0.842227	0.411052
H-L	0.709627	0.713662	0.700246	0.704782	0.704782	0.742795	1.000000	0.085662	0.714535	0.723034	0.729003	0.496956
O-C	0.020707	0.015517	0.013494	-0.033102	-0.033102	0.010311	0.085662	1.000000	-0.024535	-0.022995	-0.022155	0.009115
7 DAYS MA	0.997970	0.997977	0.997735	0.998957	0.998957	0.837809	0.714535	-0.024535	1.000000	0.999319	0.998087	0.395403
14 DAYS MA	0.996655	0.996744	0.996248	0.997559	0.997559	0.840299	0.723034	-0.022995	0.999319	1.000000	0.999521	0.388678
21 DAYS MA	0.995152	0.995310	0.994625	0.996012	0.996012	0.842227	0.729003	-0.022155	0.998087	0.999521	1.000000	0.388551
7 DAYS STD DEV	0.415781	0.417860	0.412403	0.415152	0.415152	0.411052	0.496956	0.009115	0.395403	0.388678	0.388551	1.000000

```
In [6]: f, ax = plt.subplots(figsize=(10, 12))
corr = nse_data.corr()
mask = np.triu(np.ones_like(corr, dtype=bool)) # Use bool directly
cmap = sb.diverging_palette(220, 10, as_cmap=True)
sb.heatmap(corr, mask=mask, cmap=cmap, vmax=.3, center=0, annot=True,
```

```

square=True, linewidths=.5, cbar_kws={"shrink": .7})
bottom, top = ax.get_ylim()
ax.set_ylim(bottom + 0.5, top - 0.5)
plt.show()

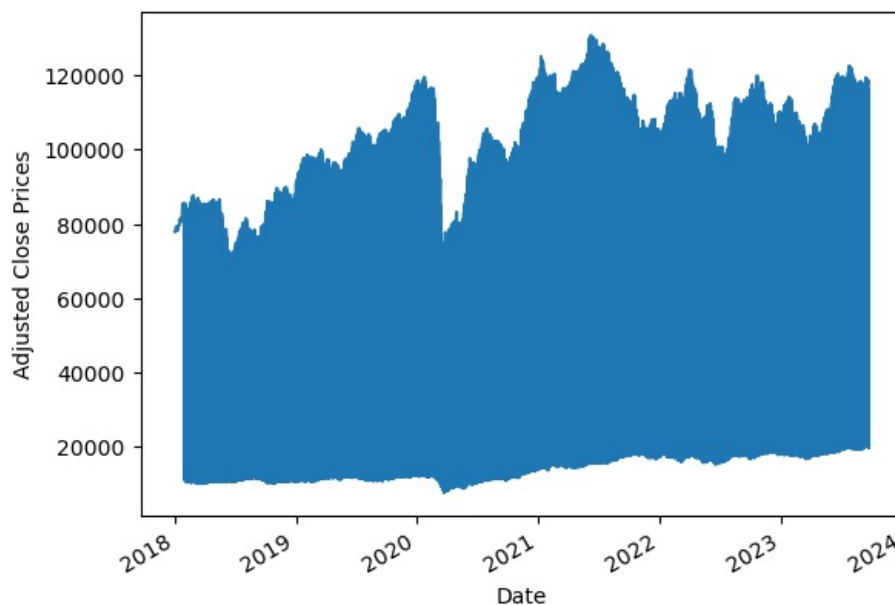
```



```

In [7]: nse_data['Adj Close'].plot()
plt.ylabel("Adjusted Close Prices")
plt.show()

```



Linear Regression

```
In [8]: # Select the "Adj Close" and "Date" columns from the stock data
adj_close = nse_data['Adj Close']
high = nse_data['High']

# Create a DataFrame with "Adj Close" and "Date" columns
data = pd.DataFrame({'Adj Close': adj_close, 'High': high})

# Add a constant term to the independent variable
data = sm.add_constant(data)

# Fit the linear regression model
model = sm.OLS(data['Adj Close'], data[['const', 'High']]).fit()

# Print the regression results
print(model.summary())
```

```

                        OLS Regression Results
=====
Dep. Variable:          Adj Close    R-squared:                0.997
Model:                  OLS         Adj. R-squared:            0.997
Method:                 Least Squares   F-statistic:            1.090e+06
Date:                  Sun, 24 Sep 2023   Prob (F-statistic):      0.00
Time:                  14:42:38         Log-Likelihood:         -25778.
No. Observations:      2813            AIC:                   5.156e+04
Df Residuals:          2811            BIC:                   5.157e+04
Df Model:               1
Covariance Type:        nonrobust
=====
               coef      std err          t      P>|t|      [0.025      0.975]
-----
const         138.1802      71.167        1.942      0.052      -1.365      277.725
High           0.9895       0.001    1044.239      0.000        0.988        0.991
=====
Omnibus:                 8588.448    Durbin-Watson:           1.092
Prob(Omnibus):            0.000    Jarque-Bera (JB):       592944108.972
Skew:                     44.670    Prob(JB):               0.00
Kurtosis:                 2250.423    Cond. No.               1.23e+05
=====
```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 1.23e+05. This might indicate that there are strong multicollinearity or other numerical problems.

Multiple Regression

```
In [9]: # Create a DataFrame with the selected columns
data = nse_data[['Open', 'High', 'Low', 'Close', 'Adj Close', 'Volume']]

# Add a constant term to the independent variables
data = sm.add_constant(data)

# Fit the multiple regression model
model = sm.OLS(data['Adj Close'], data[['const', 'Open', 'High', 'Low', 'Close', 'Volume']]).fit()

# Print the regression results
print(model.summary())
```

OLS Regression Results

```

=====
Dep. Variable:          Adj Close    R-squared:                1.000
Model:                  OLS          Adj. R-squared:          1.000
Method:                 Least Squares  F-statistic:             1.132e+31
Date:                  Sun, 24 Sep 2023  Prob (F-statistic):       0.00
Time:                  14:42:38       Log-Likelihood:          57501.
No. Observations:      2813          AIC:                    -1.150e+05
Df Residuals:          2807          BIC:                    -1.150e+05
Df Model:               5
Covariance Type:       nonrobust
=====

```

	coef	std err	t	P> t	[0.025	0.975]
const	-7.643e-11	1.01e-11	-7.539	0.000	-9.63e-11	-5.65e-11
Open	4.774e-15	9.46e-15	0.505	0.614	-1.38e-14	2.33e-14
High	-1.221e-14	1.04e-14	-1.174	0.241	-3.26e-14	8.19e-15
Low	3.331e-15	8.31e-15	0.401	0.689	-1.3e-14	1.96e-14
Close	1.0000	2.72e-15	3.67e+14	0.000	1.000	1.000
Volume	-1.084e-18	2.35e-18	-0.461	0.645	-5.7e-18	3.53e-18

```

=====
Omnibus:                11200.868    Durbin-Watson:           0.002
Prob(Omnibus):           0.000       Jarque-Bera (JB):        395.577
Skew:                    0.112       Prob(JB):                1.26e-86
Kurtosis:                1.177       Cond. No.                1.15e+07
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
 [2] The condition number is large, 1.15e+07. This might indicate that there are strong multicollinearity or other numerical problems.

Random Forest

```

In [10]: temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values

```

```

In [11]: x_train_rf, x_test_rf, y_train_rf, y_test_rf = train_test_split(x, y, test_size=0.20, random_state=0)

```

```

In [12]: scale = StandardScaler()
x_train_rf = scale.fit_transform(x_train_rf)
x_test_rf = scale.transform(x_test_rf)

```

```

In [13]: grid_rf = {
    'n_estimators': [20, 50, 100, 500, 1000],
    'max_depth': np.arange(1, 15, 1),
    'min_samples_split': [2, 10, 9],
    'min_samples_leaf': np.arange(1, 15, 2, dtype=int),
    'bootstrap': [True, False],
    'random_state': [1, 2, 30, 42]
}

# Instantiate the random forest model
model = RandomForestRegressor()

rscv = RandomizedSearchCV(estimator=model, param_distributions=grid_rf, cv=3, n_jobs=-1, verbose=2, n_iter=10,
rscv_fit = rscv.fit(x_train_rf, y_train_rf)

# Get the best parameters and best model
best_params = rscv_fit.best_params_
best_model = rscv_fit.best_estimator_

print(best_params)
print(best_model)

```

Fitting 3 folds for each of 10 candidates, totalling 30 fits

```

{'random_state': 42, 'n_estimators': 20, 'min_samples_split': 2, 'min_samples_leaf': 1, 'max_depth': 13, 'bootstrap': False}
RandomForestRegressor(bootstrap=False, max_depth=13, n_estimators=20,
random_state=42)

```

```

In [14]: model = RandomForestRegressor(n_estimators=314, random_state=42, min_samples_split=4, min_samples_leaf=3, max_d
model.fit(x_train_rf, y_train_rf)
predict = model.predict(x_test_rf)
print(predict)

```

```

[ 9853.42296294 11830.05195875 10740.86974603 113232.91774995
 11082.9085045 11355.39575099 110128.54814465 16869.36842431
 96115.30036672 104800.39713973 107234.09757186 108455.79903996
 11810.76301435 13570.78324226 75209.29151478 94622.23007407]

```

113584.99028305	10695.65857667	15747.45803188	110324.32587632
83295.72661499	82752.25002826	11936.2763322	11311.0495073
16661.39624098	12840.90307212	18781.11573797	98878.33142377
10210.43642802	10004.34326475	77526.69515604	115879.62688988
106923.54065949	102125.09575624	10550.2762329	85486.53403486
108978.91821208	17412.58897648	76207.69238089	11281.24858338
15857.95364403	10155.80468091	85485.50447011	101285.94606495
11598.88063732	17857.42014071	92161.5650206	9175.17304653
111919.96511889	17659.47375694	11006.77939764	116750.81251866
10381.38733823	13529.73949152	87399.70176774	18274.07283349
11751.06873644	111855.46788805	17313.27557618	113398.23739557
117512.75026091	110570.36660201	81735.12798146	11912.36305571
10963.64891083	119077.35528646	93634.94249881	14128.49206525
8720.48784779	10729.92545262	18193.6233691	17075.57194915
10737.52031023	102943.93967062	113780.34421178	11924.86723472
11209.71024568	112041.99275891	85776.20219299	10764.55214437
117868.28109634	12054.69603278	102242.27828158	17369.37176651
14384.52443471	83143.12864978	115171.00608332	111709.35759023
117337.2577667	11582.5526683	18606.93626722	70725.0964921
16342.61335605	94762.24016405	12093.79437571	95313.67725091
114177.38929479	119395.3135585	17767.77425768	17661.06069859
17272.43380232	113132.74819062	18292.80423727	16986.7789374
11738.16696193	98616.14371813	79361.20565838	17718.73546112
96425.30617763	108695.44975827	17529.26239531	94621.07225888
9369.59104994	10684.403847	84669.83519292	15102.1544168
17055.60047888	16592.4153912	11940.28391888	100767.42972227
115225.38705927	117184.68603787	10494.01513128	17100.44538677
105704.48653677	10820.39251046	14852.54115644	10862.13029203
10581.28643533	17220.25581502	10802.4132519	16678.74930063
104546.66021441	83814.44350006	106488.14921919	17396.41326694
103129.07407365	14690.85778988	16503.70636838	100519.12121649
9016.48353112	116671.88609381	14883.15418138	116228.73325535
10223.25238951	12173.98059044	11464.81881205	108185.91125674
118338.62576068	95876.53194261	11711.78813664	108785.31576548
18197.0413622	11758.17492497	10908.4180536	18354.50080581
11783.24055457	110244.04238375	104552.66092132	78174.67583168
16909.63204758	18108.30349847	11568.69417676	10474.17325222
11560.20684929	111497.49950655	10820.87368898	18712.50038351
100752.70354064	17042.45705166	16237.61764392	124618.34482487
129299.0369424	102288.23538446	9928.66334669	17856.83620137
79368.89188553	11244.55391566	107941.48332513	116160.07099868
98032.06382696	105616.15873775	11995.02882728	86909.65162832
112235.98140491	17621.34218859	11471.47454328	98292.99047412
101240.87300141	9286.73513474	78877.14770595	107718.92320476
97650.2147509	117932.31607131	103914.32796953	102952.39310475
12360.06310462	11556.27701668	17814.34221681	13553.32821367
96292.29320769	10949.55487141	78134.28551001	114632.23688062
10655.28879631	14908.68890206	117811.77572311	110590.02972344
9452.90731127	10564.41906657	99789.37373964	17623.22959731
10157.10514782	82920.62729548	84520.05403404	117169.95903272
102177.66361339	13985.50362648	16901.32195859	95344.07846102
11226.55877666	99353.6388558	12178.51549041	71550.72500585
102930.80192211	17275.32316464	16250.51985352	19293.01920817
17754.31413735	115165.86734643	129836.19792438	11511.88128209
17755.99152995	8225.31580071	121999.43670612	17532.1410939
11278.07927747	79021.58586433	89344.29835963	98951.16070314
110956.98391456	105854.88435136	101913.27001406	110915.43242583
122629.19082775	118730.77031498	110238.85914646	118393.11044211
10765.60372156	14832.1479061	15669.35568713	18713.32113219
10891.04228845	117871.59470846	17354.8544619	91884.78335955
10731.36785701	17931.86633663	12925.44309792	17005.83605685
104125.30690156	99081.27530273	12122.89090912	16358.04496806
83872.22471278	116963.68432877	16707.74187754	103669.65395992
81273.57675594	78610.6872742	95587.19957595	104316.83297022
79342.62407664	10307.61819792	76395.84850612	18253.63542664
84516.39164456	10128.65919274	112292.4305105	16398.58984835
86962.62909452	17101.61537488	18047.15017709	11273.45950224
16780.04481711	101450.54727691	95447.80170104	102813.13109941
88895.16620012	11304.82950713	17243.68320754	119524.74901203
10741.27159267	108217.67782357	110574.66546588	15330.8690021
11279.99964364	17604.88825854	100014.17391832	115998.10419731
11598.66263308	109905.2034323	118865.7628132	19356.38021415
84089.8090592	101010.2414934	10949.22677395	14922.00766644
106812.4758143	96580.7646946	116163.61515499	15838.10150232
114071.33452236	11914.13681766	12205.12499942	119387.76496381
17873.80669071	113429.29318423	10932.41370248	100561.14843234
11669.14616337	103768.84667898	17335.96623366	118750.7313212
15775.10978127	17309.85164469	19415.40152297	114637.26487508
18401.28107972	15593.35388606	11272.24522688	10398.98065809
11587.04854903	100608.68901722	108456.64956317	17113.80628187
85652.78454726	17723.32564564	101820.52708879	19323.70713176
12250.36655029	103184.26858979	112289.42673055	17510.38753144
115067.34563643	117258.66329486	17586.98214119	104715.18104341

17814.60639829	85524.48105128	107059.5973445	104104.75433402
12163.20346526	127210.89305266	11913.44033116	10885.67664806
118084.07774591	77753.46723712	100733.56022877	14920.9228604
17538.06921401	99174.99538538	11691.38665075	11225.56088786
109923.09740179	9321.87804794	117694.83555059	11437.17554762
109421.34045827	15770.85662374	95586.34560896	107226.74176576
17576.6900077	15776.80983717	11164.76255304	17491.85052008
79196.98588409	14566.32287045	12255.16478336	12857.24509585
85488.41207424	18196.91333976	15816.28530073	12020.25743085
107511.77110206	11727.80555467	104703.20423253	76340.73759994
85151.22922159	85348.72408676	109744.97148613	17266.23832977
15236.60965024	106466.53344992	109025.15692296	11668.50202079
112551.99093308	11388.06880876	121935.0193245	10716.23975036
76099.17657279	100566.33436526	85283.67495099	110250.66695406
72320.9482723	116920.54830153	17172.01126356	12086.02094169
17709.91768833	112898.78541519	102164.81922685	75299.05659438
11782.99055893	97892.71309901	15750.1468339	96993.41384072
8925.17511802	15823.08235036	11165.79562211	96556.28262291
17671.21122823	10903.41514833	100778.66323506	116890.4705974
18063.4585464	10606.12538191	107522.28802906	17915.6084589
114973.67547893	19429.71363338	10860.88242283	109270.94915527
10948.636018	95983.02871699	119933.7047549	10518.0342777
110909.39242275	18208.7995563	11556.19294339	107380.31731845
101218.32527747	10456.81340858	104757.36112605	120601.12942344
113671.60304384	8913.79928619	120704.10118439	104504.7366226
11130.85090772	18484.3416788	16137.31888326	119083.05518789
12093.20150984	11565.38413443	11131.5738606	103901.22292201
16171.74620342	77370.84409826	110396.14198939	10781.64233239
10707.34828963	101432.53224304	95066.97822862	112211.28644222
122943.48494993	76727.52461844	14509.58048337	10888.42993372
18426.13877426	101242.69311692	106348.13739855	108209.5090009
10425.63775339	94286.74470306	103693.29670247	17892.14261917
9470.47162991	14290.76111316	127298.35686193	11893.59130501
108197.32646627	116183.9404676	79787.37326036	19389.13550683
17826.326865	10246.80088735	118942.86966619	85218.63368158
83056.16290062	83151.96497826	95346.31571458	17154.02880794
17650.18069517	74399.00070944	107346.58457151	10738.63223617
11245.36888422	16977.64631376	104798.47273543	93534.95281247
12745.88406587	129991.07584243	17324.93251635	12213.39305157
10526.85565596	108844.02115663	9930.41257369	15312.20137733
101232.72351217	18393.22029108	16109.70822411	98017.02299692
114301.17186859	113784.03482951	103264.32322267	17010.8812106
19561.97037458	17325.14163372	79383.42214157	11409.84602645
14562.074884	10199.88146253	11890.09110306	74754.53291422
120555.21102288	12782.29562047	11724.57289666	12144.36378409
114973.05735984	120226.24147963	121611.49736641	82773.88757249
102792.43860371	107212.99737311	85619.52610479	125893.60577695
82065.90013775	14495.52539874	10481.34023684	84243.36806436
17621.83233355	9054.21376327	109079.6373438	12870.42443111
17344.43408724	11508.71580351	18203.5364655	17623.68915146
117345.11914631	114828.45390779	17950.18887587	113696.5884485
93881.00166474	11470.42141291	111894.31709256	10617.55542435
101809.74098526	98282.91881428	105018.97749478	98298.51314096
103913.04579565	9792.88991761	11437.52532545]	

```
In [15]: print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_rf, predict), 4))
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_rf, predict), 4))
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_rf, predict)), 4))
print("(R^2) Score:", round(metrics.r2_score(y_test_rf, predict), 4))
print(f'Train Score : {model.score(x_train_rf, y_train_rf) * 100:.2f}% and Test Score : {model.score(x_test_rf,
errors = abs(predict - y_test_rf)
mape = 100 * (errors / y_test_rf)
accuracy = 100 - np.mean(mape)
print('Accuracy:', round(accuracy, 2), '%.')
```

Mean Absolute Error: 93.2655

Mean Squared Error: 2789568.5694

Root Mean Squared Error: 1670.2002

(R^2) Score: 0.9987

Train Score : 100.00% and Test Score : 99.87% using Random Tree Regressor.

Accuracy: 99.88 %.

```
In [16]: predictions = pd.DataFrame({"Predictions": predict})
predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_rf_NSE.csv")
#collects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_rf_NSE.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_rf_NSE.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_rf_NSE.csv")
```

```
In [17]: oneyear_df_pred = pd.read_csv("one-year-predictions_rf_NSE.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year Random Forest", color=
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date

Predictions

Date

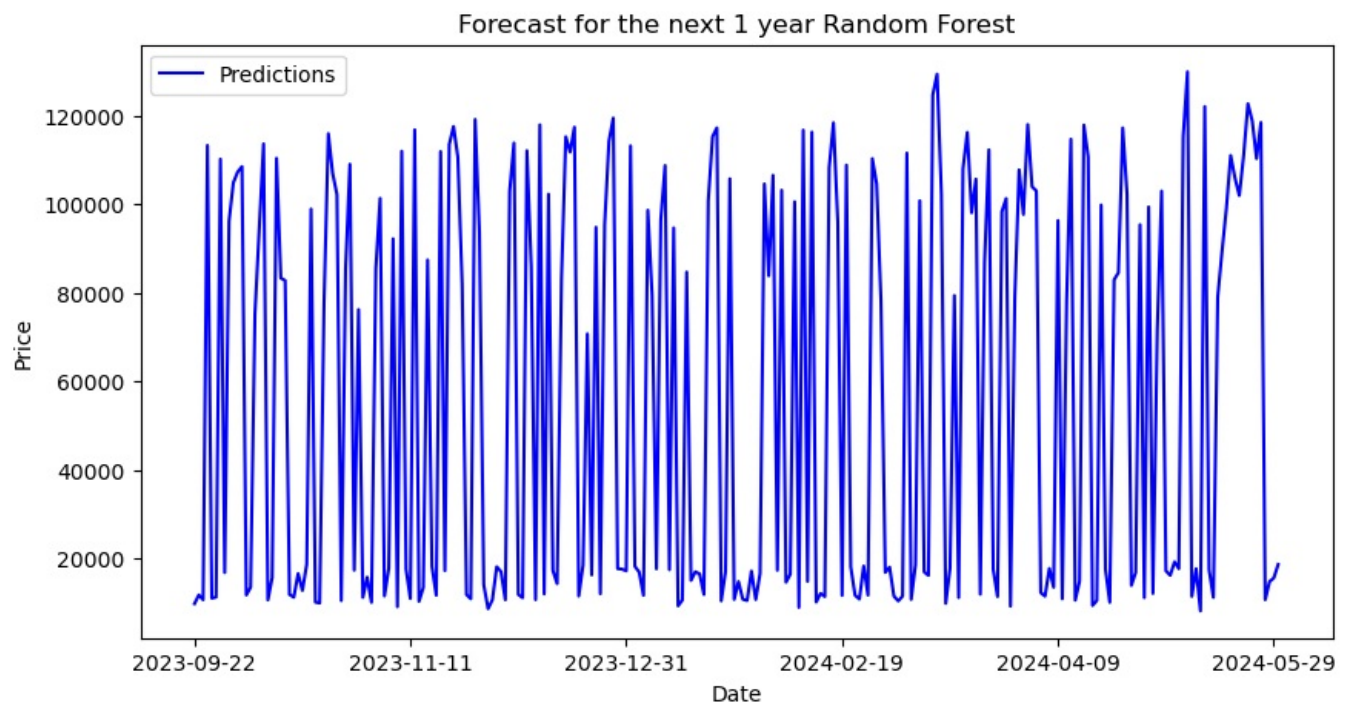
2024-05-12 8225.315801

Sell price and date

Predictions

Date

2024-05-09 129836.197924



```
In [18]: onemonth_df_pred = pd.read_csv("one-month-predictions_rf_NSE.csv")
onemonth_df_pred.set_index("Date", inplace=True)
buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
print(onemonth_sell)
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month Random Forest", colo
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date

Predictions

Date

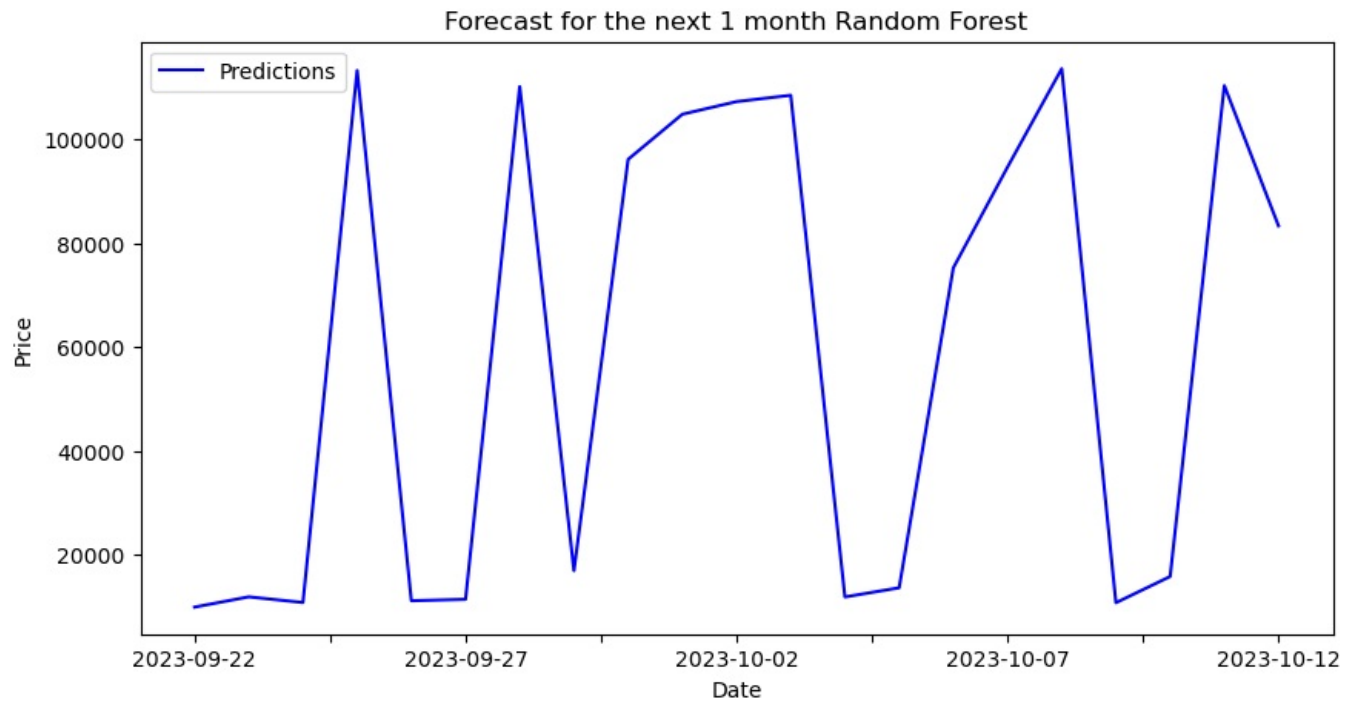
2023-09-22 9853.422963

Sell price and date

Predictions

Date

2023-10-08 113584.990283



```
In [19]: fivedays_df_pred = pd.read_csv("five-days-predictions_rf_NSE.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days Random Forest", color='blue')
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date

Predictions

Date

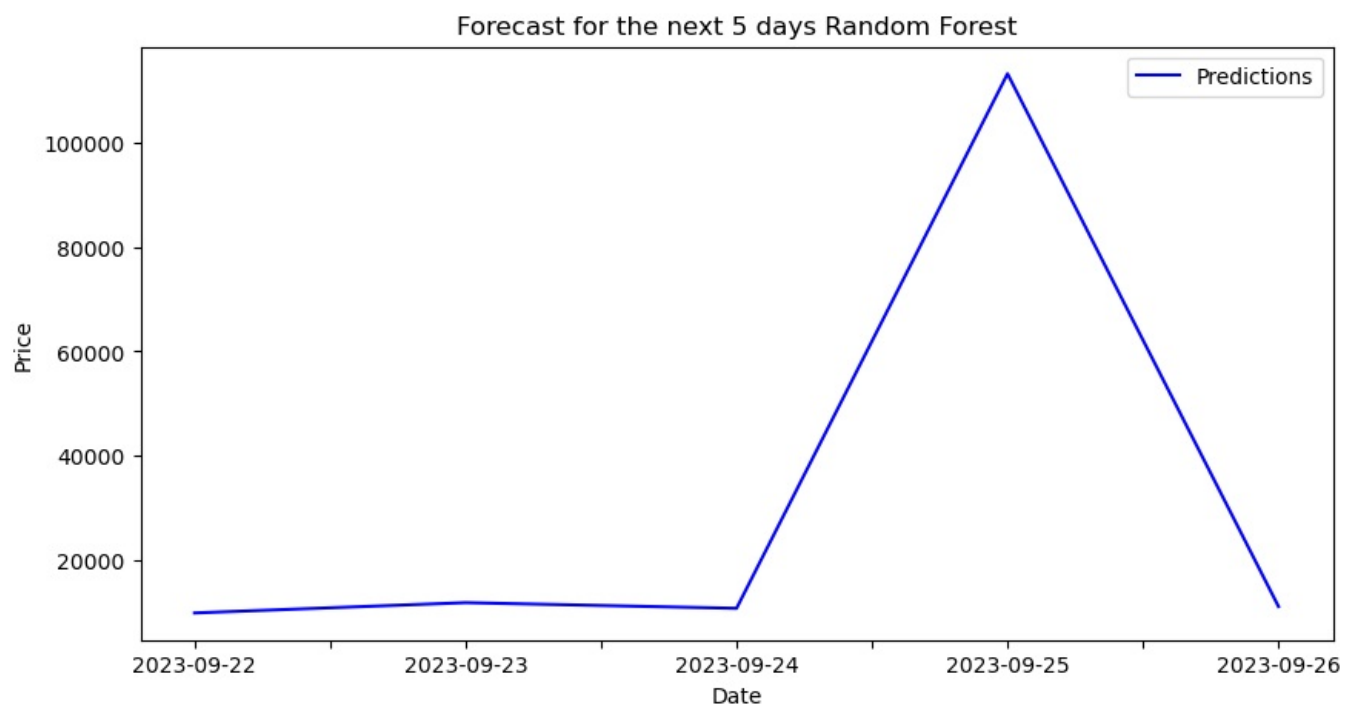
2023-09-22 9853.422963

Sell price and date

Predictions

Date

2023-09-25 113232.91775



SVM

```
In [20]: temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values

In [21]: x_train_svm, x_test_svm, y_train_svm, y_test_svm = train_test_split(x, y, test_size=0.20, random_state=0)

In [22]: scale = StandardScaler()
x_train_svm = scale.fit_transform(x_train_svm)
x_test_svm = scale.transform(x_test_svm)

In [23]: # Define the parameter grid for hyperparameter tuning
param_grid = {
    'C': [0.1, 1.0, 10.0],
    'gamma': [0.1, 1.0, 'scale'],
    'kernel': ['linear', 'rbf']
}

# Instantiate the SVM model
svm_model = SVR()

# Perform grid search with cross-validation
grid_search = GridSearchCV(estimator=svm_model, param_grid=param_grid, cv=5, scoring='neg_mean_squared_error')
grid_search.fit(x_train_svm, y_train_svm)

# Get the best parameters and best model
best_params = grid_search.best_params_
```

```
best_model = grid_search.best_estimator_

print(best_params)
print(best_model)

{'C': 10.0, 'gamma': 0.1, 'kernel': 'linear'}
SVR(C=10.0, gamma=0.1, kernel='linear')
```

```
In [24]: # Retrain the best model on the full training set
best_model.fit(x_train_svm, y_train_svm)
```

```
# Make predictions
predict = best_model.predict(x_test_svm)
predict
```

```
Out[24]: array([ 9917.87570183, 11886.99223766, 10797.06437594, 113108.03658749,
11105.78363912, 11358.39020968, 108119.15862039, 17138.43981822,
95469.15982445, 105246.30255316, 105696.86888052, 109990.08865652,
11952.97470112, 13521.38443558, 75658.06794587, 94617.80195607,
112217.92816704, 10792.12661821, 15729.54503147, 109754.2492531 ,
83079.17368976, 83185.12448244, 11976.29808605, 11523.54957068,
16564.44279402, 12771.704872 , 18630.57873923, 98986.86282677,
10461.1939279 , 10096.4701261 , 76656.09843736, 113798.70005735,
106930.41566915, 102435.78732752, 10679.5632153 , 84322.80496142,
109168.29226313, 17449.83626843, 76227.4321452 , 11431.60776321,
15780.91588867, 10335.21346711, 85195.38104545, 101298.66665028,
11607.87729291, 17691.83072586, 92799.84453869, 9399.45055366,
110663.21422404, 17473.41157922, 11038.44801689, 115504.68029675,
10551.44030209, 13452.01243937, 86593.14416431, 18134.69352548,
11807.07063862, 110071.70859811, 17231.79988816, 112685.19041724,
117602.13127577, 109142.15597154, 81733.86306053, 11987.92467241,
11024.06987251, 117431.10761104, 91685.56561753, 14074.62513956,
9153.5908398 , 10843.76845192, 18229.8857351 , 16923.75414402,
10893.72495305, 101837.94784194, 114149.56447012, 11938.66607899,
11286.72505416, 112728.37679809, 85443.30891849, 10769.846927 ,
116497.12511716, 12170.38996856, 101315.68222994, 17320.90186279,
14438.3833974 , 83251.63539949, 118440.01461298, 111639.71120903,
116039.41757538, 11784.45858186, 18516.35314769, 70521.38901237,
16274.47869651, 94302.39061304, 12263.42888695, 96723.30290627,
113291.18856044, 118256.90502733, 17784.7713221 , 17503.4973913 ,
17355.05175963, 112100.20895004, 18101.52626315, 17060.25350179,
11824.69919215, 98719.71701731, 78989.77129224, 17726.41921621,
95585.2862572 , 107402.54409535, 17528.66553985, 96293.70193353,
9462.79414009, 10710.29106226, 83635.1253809 , 15057.91868524,
17265.81079378, 16638.55044068, 11998.66429961, 101438.1592914 ,
114402.38216101, 116970.57373351, 10642.38890819, 17430.72775277,
107460.54986522, 10948.28520114, 14886.15162696, 10907.34234983,
10648.16671944, 17212.49244348, 10852.72029961, 16688.98073154,
104810.92138098, 82766.48143716, 105395.36552153, 17512.36496636,
102611.61697798, 14761.40252873, 16730.04345044, 99845.503484 ,
9250.05457273, 115494.50711106, 14880.07255581, 116012.42939491,
10444.252385 , 12224.05065745, 11449.50310056, 107915.17895141,
118004.9361139 , 95519.80452836, 12069.66566932, 108222.52426424,
18012.53139794, 11754.53534321, 10990.15993504, 18217.67501225,
11814.06451128, 112480.74617944, 103956.34874849, 77318.80505823,
16827.49857676, 18058.51435109, 11669.00784174, 10574.06951179,
11737.49920124, 110341.29386184, 10854.40170667, 18604.5234933 ,
101067.19812296, 17124.94939615, 16333.54786609, 123821.48005271,
128270.45352628, 101937.59595862, 10070.77104665, 17760.66688755,
79929.48880129, 11333.25391418, 107620.70703349, 118131.32265424,
102112.8431984 , 107303.94945036, 12066.42364884, 87008.24891821,
111035.84760287, 17535.42751334, 11587.71787793, 98532.10378063,
101394.33143423, 9550.73691388, 80304.06669401, 108173.51799896,
96327.42910835, 116613.36619798, 104258.76159012, 104319.34104535,
12396.70831942, 11605.88797074, 17683.82378368, 13525.02888268,
94986.73017656, 11002.90288932, 77057.45596921, 116827.76581692,
10817.87381811, 14874.21952637, 116430.96418444, 108859.49144418,
9661.56907771, 10577.93380114, 99054.03963519, 17490.12346869,
10385.8497039 , 81497.7535784 , 84155.01407106, 117050.33095134,
102092.80781686, 13985.3209929 , 16788.58896502, 94821.77762849,
11496.54968626, 98387.21027618, 12206.63953973, 71747.43969737,
102319.77340758, 17175.85149321, 16506.44422778, 19251.18859361,
17795.82269948, 114223.58606692, 128351.70530403, 11617.73556132,
17790.28571248, 8902.57388517, 121625.1645508 , 17288.12447691,
11637.83910139, 78137.10668103, 88502.66161946, 99908.30365013,
112189.62209026, 105395.05837354, 103430.09144001, 110995.78082663,
121343.59035209, 116940.62823221, 109045.6278245 , 118435.92165959,
10811.42724027, 14991.26824227, 15673.64033312, 18538.84327315,
10961.91230365, 117648.63728808, 17269.77399616, 91963.05359788,
10799.55404116, 18039.86347935, 12987.62336955, 17052.82947173,
103628.53779565, 100126.43240519, 12200.9385883 , 16229.91982113,
83283.6986297 , 117003.86581752, 17147.82928587, 104749.65891422,
75899.28909032, 77679.22676633, 98316.66433263, 103005.36340583,
70712.35707972, 10734.0360384 , 133613.91879401, 18171.79246179,
```

82765.78221903, 10214.33385463, 112415.487979 , 16756.33243872,
85895.08861218, 17119.18868377, 17973.92268976, 11517.25547722,
16941.84770724, 101800.53289338, 94165.54324634, 103077.73116356,
87923.24919724, 11359.96919539, 17082.80673586, 118096.08518376,
10956.42248092, 106750.95961221, 109409.85151196, 15703.4363244 ,
11502.76320044, 17721.63067202, 100731.7181096 , 116461.3895878 ,
11683.9754541 , 111817.51042343, 116925.25912462, 19322.88119062,
83925.23224587, 100829.73530852, 11106.99049243, 14908.05545058,
105815.31088245, 98889.2857643 , 116642.77787817, 15809.73327582,
115090.38063178, 11939.68867822, 12250.29955627, 118997.42715708,
17776.35287353, 113306.18445654, 10961.22409459, 100735.72566615,
11701.68405652, 104571.16945212, 17152.66045105, 117465.47129874,
15812.74990215, 17337.2107923 , 19378.60908066, 113740.89857344,
18250.34855732, 15865.92201937, 11342.57284743, 10543.97693801,
11685.66526883, 100377.15781436, 110238.03573639, 17505.86168391,
85785.56070282, 17689.00425881, 101214.59543787, 19251.81079858,
12286.09548571, 103350.87149294, 111705.28844523, 17796.59756986,
115114.23610676, 116835.78820722, 17442.57069403, 105855.39784777,
17629.50413502, 85653.62556898, 107119.34012234, 102785.24953504,
12358.36051135, 126689.76586007, 11930.84423454, 10970.47052376,
118967.13306078, 78898.74182516, 99925.63955556, 15044.31254629,
17516.78501092, 96984.32346173, 11707.4442609 , 11364.34970691,
110577.63765722, 9367.18646452, 116750.54196689, 11525.09348682,
108829.65768783, 15719.71318532, 94962.29677429, 109779.15494789,
17660.51507763, 16181.39688951, 11206.60419909, 17321.26418034,
77931.0348464 , 14746.84959521, 12371.43710013, 12831.44194472,
85050.69853695, 18178.31626229, 15795.14726762, 11990.08562875,
105580.4120979 , 11843.82691801, 103480.07258008, 76610.16686174,
84407.75748245, 84944.45514712, 111985.17138644, 17367.86880943,
15111.60909046, 105713.16373487, 107690.44023051, 11829.9203735 ,
110854.09444022, 11434.35568922, 120652.51464777, 10740.65344216,
76311.11165199, 99050.46183613, 84922.68951557, 113489.22090019,
74931.1064618 , 117509.78546213, 17278.84019957, 12160.09842864,
17697.38532636, 111642.19610114, 102425.45758861, 75322.26064665,
11882.85598885, 98247.35531037, 15766.54714054, 96190.96938781,
9251.94586476, 15833.7311432 , 11219.97856506, 95773.32772833,
17560.44776399, 11017.26595248, 102767.68780767, 116654.5653602 ,
17939.13268822, 10687.41376381, 107338.64524591, 17893.80439443,
113258.19552688, 19330.31872881, 11137.75207338, 109133.46654577,
10973.28242122, 95746.75130059, 119149.80438248, 10691.6854016 ,
112220.67345304, 18109.7549735 , 11563.87075019, 106942.97899177,
102235.0325424 , 11231.42386842, 103393.13102059, 119061.51685794,
113548.21356279, 9252.24314778, 120697.27925368, 102999.98913606,
11135.29804389, 18382.70436203, 16212.42798826, 117982.72581565,
12165.71627286, 11565.76785924, 11397.78653333, 107055.12698634,
16139.13183604, 75972.85478242, 110620.99201134, 10907.28147029,
10922.03595724, 100961.44483047, 93954.678067 , 110805.13808182,
121639.40949331, 77171.7997597 , 14550.5694608 , 10958.10795193,
18216.67761944, 99835.43626121, 106142.33228051, 106930.71559171,
10533.17287236, 93478.49706084, 102124.88642814, 17811.19706013,
9772.56045353, 14328.94733271, 127529.23492712, 11944.75782114,
107896.70826403, 116228.74599542, 78536.91350335, 19417.93135169,
17897.83539589, 10409.7075641 , 117954.19143196, 84762.60796199,
83343.02598089, 82741.01355046, 95596.80613057, 17085.55130653,
17848.71440138, 87550.40619431, 108172.69973019, 10843.84458243,
11397.08190782, 17202.27164442, 103376.58209176, 92259.55805336,
12770.86729721, 127986.01001307, 17368.85237944, 12266.09102028,
10552.90508214, 109090.43954249, 10073.15150322, 15166.50827448,
101879.8754319 , 18386.24353216, 16001.24531399, 97685.7198941 ,
112996.63676115, 112067.92167863, 105508.0531234 , 17290.79628336,
19419.64065401, 17361.56111837, 80813.37518417, 11559.32843164,
14550.25569098, 10274.31683634, 11951.45035316, 80668.40394722,
119644.64255419, 12691.57159197, 11818.13914363, 12188.8544618 ,
116390.87787773, 119813.10358648, 120301.79030095, 83698.97485846,
103893.20122753, 105378.38920305, 84544.80331331, 125745.82809581,
81666.1268243 , 14578.51957464, 10524.30954406, 83599.37603873,
17706.90475327, 10108.84148909, 109796.90667761, 12885.876239 ,
17190.40336092, 11746.69593505, 17965.00331708, 17532.34955171,
116542.00801904, 114460.0423439 , 17742.88903361, 112362.70741374,
93189.5002576 , 11536.7892459 , 111612.95686999, 10657.00294157,
101436.72117916, 99142.62680482, 103706.90890816, 97389.44312038,
103544.40556244, 9953.97060998, 11551.70762978])

```
In [25]: print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_svm, predict), 4))
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_svm, predict), 4))
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_svm, predict)), 4))
print("(R^2) Score:", round(metrics.r2_score(y_test_svm, predict), 4))
print(f'Train Score : {model.score(x_train_svm, y_train_svm) * 100:.2f}% and Test Score : {model.score(x_test_svm, y_test_svm) * 100:.2f}%')
errors = abs(predict - y_test_svm)
mape = 100 * (errors / y_test_svm)
accuracy = 100 - np.mean(mape)
print('Accuracy:', round(accuracy, 2), '%.')
```

Mean Absolute Error: 584.4531
Mean Squared Error: 1718798.776
Root Mean Squared Error: 1311.0297
(R^2) Score: 0.9992
Train Score : 100.00% and Test Score : 99.87% using Support Vector Regressor.
Accuracy: 99.02 %.

```
In [26]: predictions = pd.DataFrame({"Predictions": predict})
predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_svm_NSE.csv")
#collects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_svm_NSE.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_svm_NSE.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_svm_NSE.csv")
```

```
In [27]: oneyear_df_pred = pd.read_csv("one-year-predictions_svm_NSE.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year Support Vector Machine")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date

Predictions

Date

2024-05-12 8902.573885

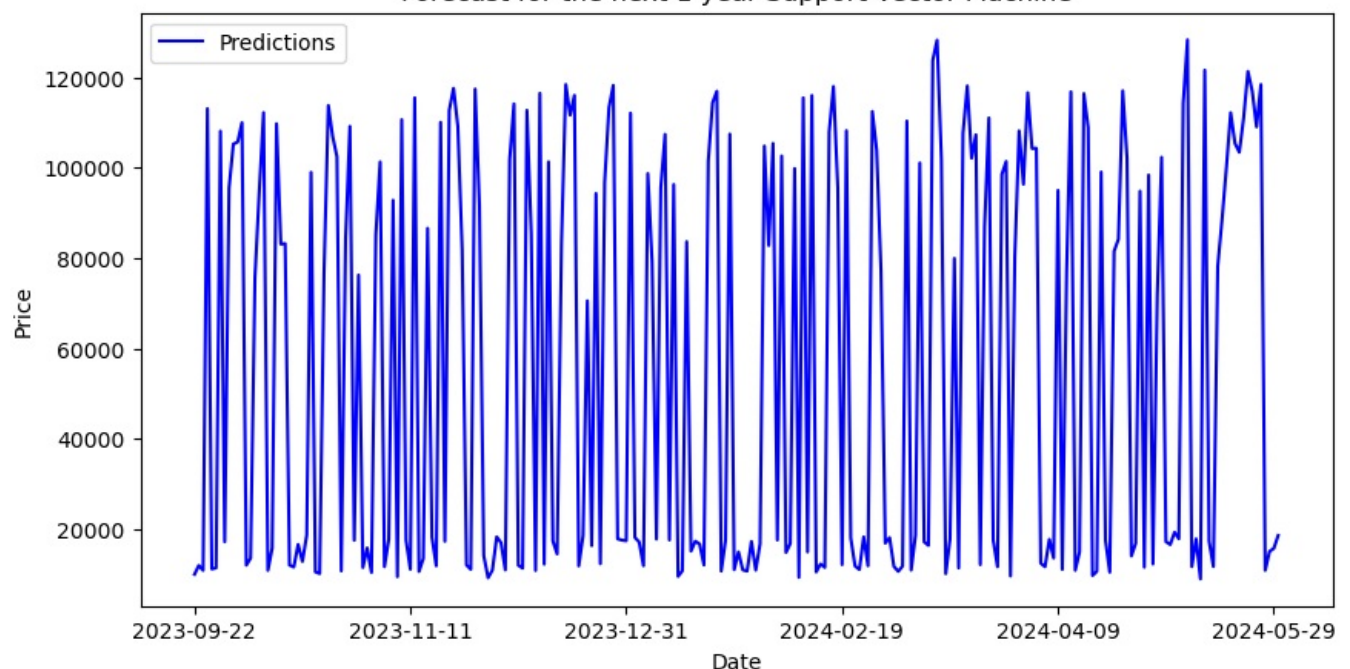
Sell price and date

Predictions

Date

2024-05-09 128351.705304

Forecast for the next 1 year Support Vector Machine



```
In [28]: onemonth_df_pred = pd.read_csv("one-month-predictions_svm_NSE.csv")
onemonth_df_pred.set_index("Date", inplace=True)
buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
print(onemonth_sell)
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month Support Vector Machine")
plt.xlabel("Date")
```

```
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date

Predictions

Date

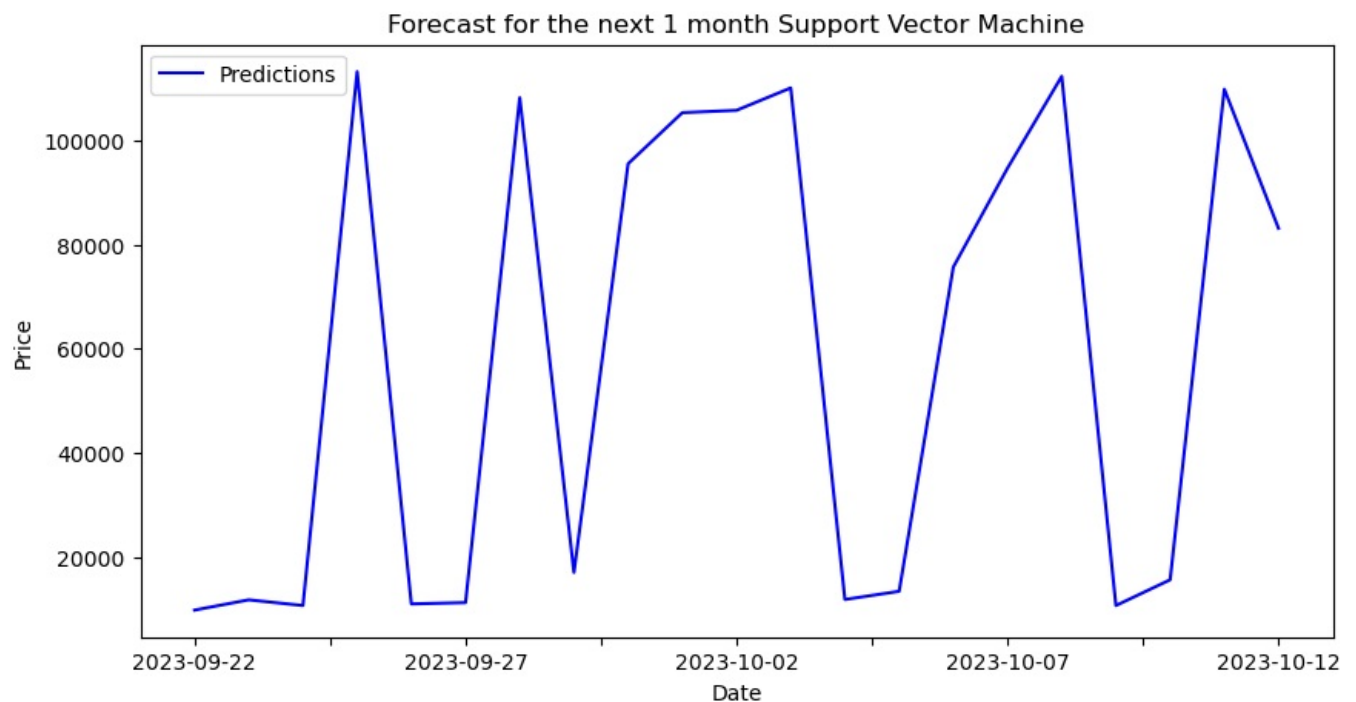
2023-09-22 9917.875702

Sell price and date

Predictions

Date

2023-09-25 113108.036587



```
In [29]: fivedays_df_pred = pd.read_csv("five-days-predictions_svm_NSE.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days Support Vector Machine")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date

Predictions

Date

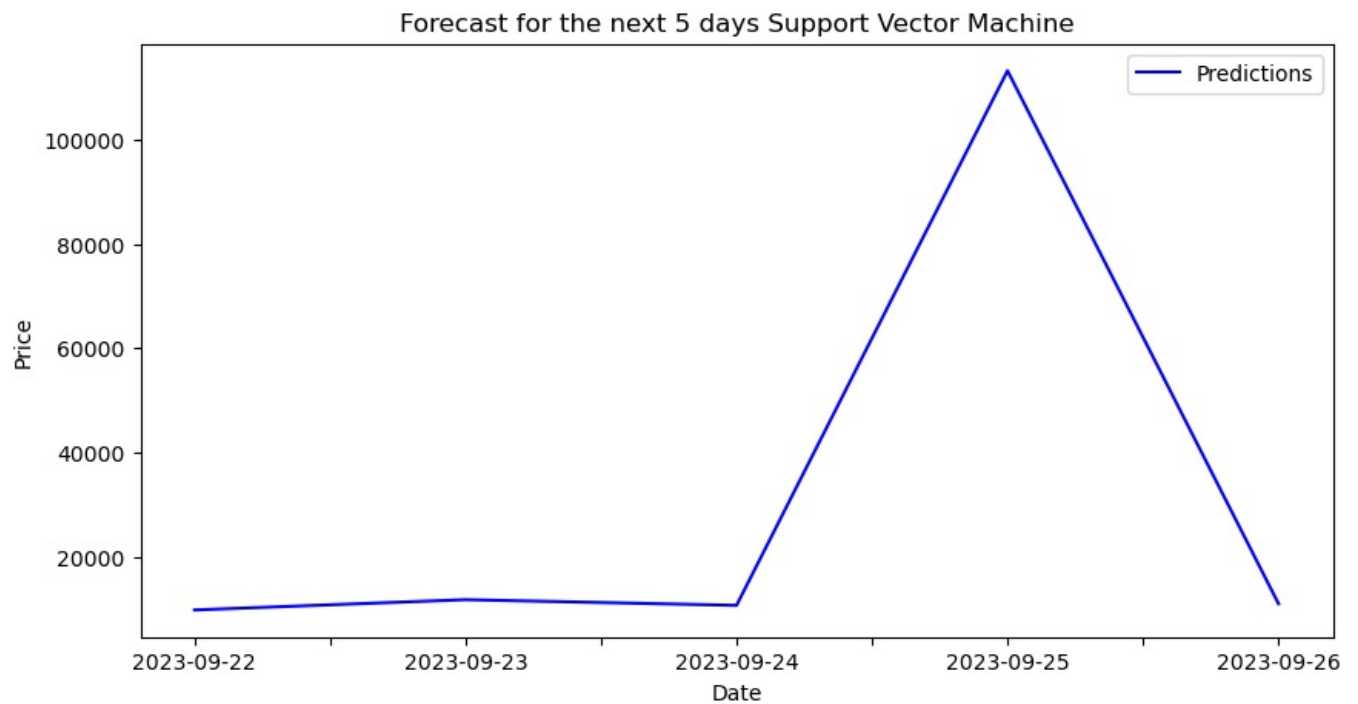
2023-09-22 9917.875702

Sell price and date

Predictions

Date

2023-09-25 113108.036587



Decision Tree

```
In [30]: temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values

In [31]: x_train_dt, x_test_dt, y_train_dt, y_test_dt = train_test_split(x, y, test_size=0.20, random_state=0)

In [32]: scale = StandardScaler()
x_train_dt = scale.fit_transform(x_train_dt)
x_test_dt = scale.transform(x_test_dt)

In [33]: # Define the parameter grid for hyperparameter tuning
param_grid = {
    'max_depth': [None, 5, 10, 20],
    'min_samples_split': [2, 5, 10],
    'min_samples_leaf': [1, 2, 4]
}

# Instantiate the Decision Tree model
dt_model = DecisionTreeRegressor()

# Perform grid search with cross-validation
grid_search = GridSearchCV(estimator=dt_model, param_grid=param_grid, cv=5, scoring='neg_mean_squared_error')
grid_search.fit(x_train_dt, y_train_dt)

# Get the best parameters and best model
best_params = grid_search.best_params_
best_model = grid_search.best_estimator_

print(best_params)
print(best_model)
```



```
{'max_depth': None, 'min_samples_leaf': 2, 'min_samples_split': 2}  
DecisionTreeRegressor(min_samples_leaf=2)
```

```
In [34]: # Retrain the best model on the full training set  
best_model.fit(x_train_dt, y_train_dt)  
  
# Make predictions  
predict = best_model.predict(x_test_dt)  
predict
```

```
Out[34]: array([[ 9975.38346354, 11819.70019531, 10740.22460938, 113223.5 ,  
11090.32519531, 11355.01660156, 110116. , 16899.79980469,  
96109. , 104666.66666667, 107340.5 , 108480.66666667,  
11819.70019531, 13597.94986979, 75440.33333333, 94596. ,  
113578.33333333, 10694.95019531, 15747.47509766, 110334. ,  
83408. , 82589. , 11930.01660156, 11311.53352865,  
16650.10058594, 12866.30029297, 18756.56640625, 98866.5 ,  
10243.52539062, 10046.06673177, 77513.33333333, 115911.5 ,  
106809.5 , 102132.5 , 10551.28352865, 85475. ,  
108963. , 17399.85058594, 76155. , 11285.89990234,  
15859.8499349 , 10124.625 , 85475. , 101346.5 ,  
11597.71679688, 17854.57519531, 92112.66666667, 9202.32470703,  
111887.66666667, 17642.77539062, 11008.17480469, 116870.33333333,  
10384.79980469, 13497.69970703, 87443.33333333, 18267.82519531,  
11761.61686198, 111827.5 , 17313.77539062, 113410. ,  
117543.5 , 110568.33333333, 81838. , 11914.01692708,  
10962.11653646, 119061. , 93675. , 14139.57519531,  
8772.48339844, 10733. , 18201.25 , 17070.84960938,  
10733. , 102933.5 , 113853.5 , 11922.33333333,  
11209.14990234, 112022.66666667, 85762.5 , 10765.65039062,  
117898.5 , 12054.31673177, 102227.33333333, 17372. ,  
14333.125 , 83408. , 115196. , 111556.5 ,  
117169.33333333, 11582.56640625, 18613.02539062, 70960. ,  
16348.89990234, 94720.5 , 12099.52490234, 95343.33333333,  
114044.5 , 119392. , 17782.17480469, 17642.77539062,  
17276.125 , 113054. , 18267.82519531, 16984.57519531,  
11738.67480469, 98597. , 79377. , 17730. ,  
96470. , 108658.66666667, 17527.97460938, 94596. ,  
9571.75 , 10685.125 , 84660.5 , 15117.375 ,  
17058.59960938, 16584.42578125, 11944.6500651 , 100739.33333333,  
115170. , 117169.33333333, 10495.54980469, 17101.30078125,  
105695. , 10830.56673177, 14859.04980469, 10866.5 ,  
10583.64990234, 17244.81705729, 10802.42529297, 16650.10058594,  
104505.33333333, 83802.5 , 106500. , 17390.42480469,  
103268.66666667, 14702.14990234, 16490.14941406, 100517.66666667,  
9052.89990234, 116668. , 14900.85009766, 116234.5 ,  
10230.75 , 12174.05013021, 11464.96679688, 108168.5 ,  
118379.5 , 95938. , 11711.51660156, 108785.5 ,  
18179.92480469, 11761.61686198, 10906.95019531, 18406.52539062,  
11761.61686198, 110269. , 104530.66666667, 78351.5 ,  
16956.01627604, 18108.64941406, 11570.60009766, 10476.5999349 ,  
11556.5 , 111458. , 10818.96647135, 18694.57519531,  
100825. , 17058.59960938, 16239.125 , 125074. ,  
129650.5 , 102301. , 9975.38346354, 17856.875 ,  
79377. , 11263.04980469, 107898. , 116155.5 ,  
98027.5 , 105611.5 , 11988.4000651 , 86937.33333333,  
112251.33333333, 17624.22558594, 11464.96679688, 98331.66666667,  
101346.5 , 9292.40039062, 78858.66666667, 107705.66666667,  
97634. , 117955. , 103909.5 , 102933.5 ,  
12342.78320312, 11556.5 , 17814.47460938, 13597.94986979,  
96333. , 10947.22509766, 78117.5 , 114598. ,  
10646.57519531, 14900.85009766, 117898.5 , 110598.5 ,  
9571.75 , 10559.79980469, 99788. , 17624.22558594,  
10124.625 , 82913. , 84536. , 117200.5 ,  
102172.33333333, 14139.57519531, 16956.01627604, 95367. ,  
11232.14973958, 99179.5 , 12174.05013021, 71413. ,  
102917.5 , 17276.125 , 16261.06673177, 19321.50065104,  
17756.42480469, 115126. , 129650.5 , 11497.18326823,  
17756.42480469, 8278.36653646, 122121. , 17534.15039062,  
11285.89990234, 79000.5 , 89265. , 98940.66666667,  
110908. , 105767.5 , 101936. , 110908. ,  
122554. , 118565.5 , 110216.33333333, 118317. ,  
10754.78320312, 14859.04980469, 15675.125 , 18694.57519531,  
10891.8499349 , 117898.5 , 17360.92480469, 91628.66666667,  
10740.22460938, 17927.54980469, 12947.70019531, 17009.15039062,  
104123. , 99043.5 , 12131.33333333, 16386.29980469,  
83903.5 , 117026. , 16745.14941406, 103718.33333333,  
81355.33333333, 78626.33333333, 95606. , 104276. ,  
79377. , 10295.5 , 18914.47460938, 18244.15039062,  
84536. , 10135.28352865, 112290.5 , 16348.89990234,  
86937.33333333, 17078.79980469, 18047.82421875, 11251.57519531,  
16899.79980469, 101541. , 95537.5 , 102779. ,  
88711.66666667, 11301.18326823, 17222.375 , 119556.5 ,
```


10737.59960938,	108259.66666667,	110568.33333333,	15302.80013021,
11285.89990234,	17603.31640625,	100002.5,	115999.
11602.14990234,	109912.	118916.	19351.67480469,
84062.5,	101015.33333333,	10947.22509766,	14900.85009766,
106809.5,	96551.33333333,	116146.	15831.38313802,
114044.5,	11914.01692708,	12215.22509766,	119392.
17872.64941406,	113410.	10932.39990234,	100517.66666667,
11671.47509766,	103775.	17335.82519531,	118565.5,
15768.33333333,	17307.875,	19436.97558594,	114634.33333333,
18417.67480469,	15579.5,	11263.04980469,	10406.57519531,
11588.72460938,	100582.	108437.5,	17120.59960938,
85647.	17730.	101822.	19351.67480469,
12257.44986979,	103268.66666667,	112290.5,	17511.60026042,
115024.5,	117200.5,	17591.97460938,	104666.66666667,
17814.47460938,	85542.66666667,	106990.	104042.5,
12174.05013021,	127342.5,	11914.01692708,	10880.19986979,
118078.33333333,	77910.5,	100739.33333333,	14925.63346354,
17534.15039062,	99179.5,	11691.47509766,	11232.14973958,
109912.	9382.22509766,	117709.5,	11439.41634115,
109482.5,	15768.33333333,	95606.	107340.5,
17576.57519531,	15768.33333333,	11157.63346354,	17488.82421875,
79153.5,	14547.80013021,	12257.44986979,	12866.30029297,
85475.	18201.25,	15812.96647135,	12005.57470703,
107523.66666667,	11711.51660156,	104666.66666667,	76392.66666667,
85066.	85284.5,	109815.5,	17276.125,
15227.02490234,	106500.	109056.	11659.44970703,
112581.33333333,	11381.54980469,	121717.	10715.44986979,
76155.	100517.66666667,	85284.5,	110269.
72184.33333333,	116976.	17170.27539062,	12099.52490234,
17709.14941406,	112896.66666667,	102172.33333333,	75075.66666667,
11761.61686198,	97906.	15751.85009766,	96995.
8966.86653646,	15831.38313802,	11157.63346354,	96551.33333333,
17673.29980469,	10906.95019531,	100825.	116870.33333333,
18069.20052083,	10605.61653646,	107650.66666667,	17914.60058594,
114980.	19436.97558594,	10866.5,	109265.33333333,
10950.	95987.	119927.	10532.52490234,
110921.	18179.92480469,	11556.5,	107414.5,
101346.5,	10459.03320312,	104666.66666667,	120689.5,
113853.5,	9052.89990234,	120629.5,	104471.5,
11113.82519531,	18506.04980469,	16141.	119061.
12081.14990234,	11570.60009766,	11129.10009766,	103909.5,
16212.36653646,	77159.	110431.33333333,	10783.21679688,
10708.10009766,	101492.	95079.5,	112251.33333333,
122972.	77159.	14547.80013021,	10880.19986979,
18345.41666667,	101225.5,	106369.5,	108207.
10430.19970703,	94275.	103649.5,	17895.02441406,
9571.75,	14333.125,	127531.	11895.95019531,
108207.	116176.5,	79712.5,	19386.66666667,
17830.67480469,	10245.125,	118916.	85232.66666667,
83100.	83408.	95367.	17151.04980469,
17663.97558594,	74648.5,	107340.5,	10737.59960938,
11263.04980469,	16979.27441406,	104836.66666667,	93675.
12675.52490234,	130492.	17326.32519531,	12215.22509766,
10525.6500651,	108884.	9975.38346354,	15302.80013021,
101225.5,	18417.67480469,	16141.	98027.5,
114310.	113853.5,	103268.66666667,	17009.15039062,
19567.67480469,	17326.32519531,	79322.	11398.92529297,
14547.80013021,	10205.10009766,	11883.78352865,	75440.33333333,
120447.	12675.52490234,	11711.51660156,	12131.33333333,
114980.	120199.	121841.	82589.
102779.	107199.33333333,	85608.5,	125414.5,
82589.	14490.625,	10476.5999349,	84214.5,
17627.125,	9052.89990234,	109056.	12947.70019531,
17335.82519531,	11497.18326823,	18179.92480469,	17627.125,
117169.33333333,	114837.66666667,	17956.54980469,	113853.5,
94008.	11464.96679688,	111930.	10617.71679688,
101822.	98279.	105018.	98331.66666667,
103909.5,	9975.38346354,	11439.41634115)	

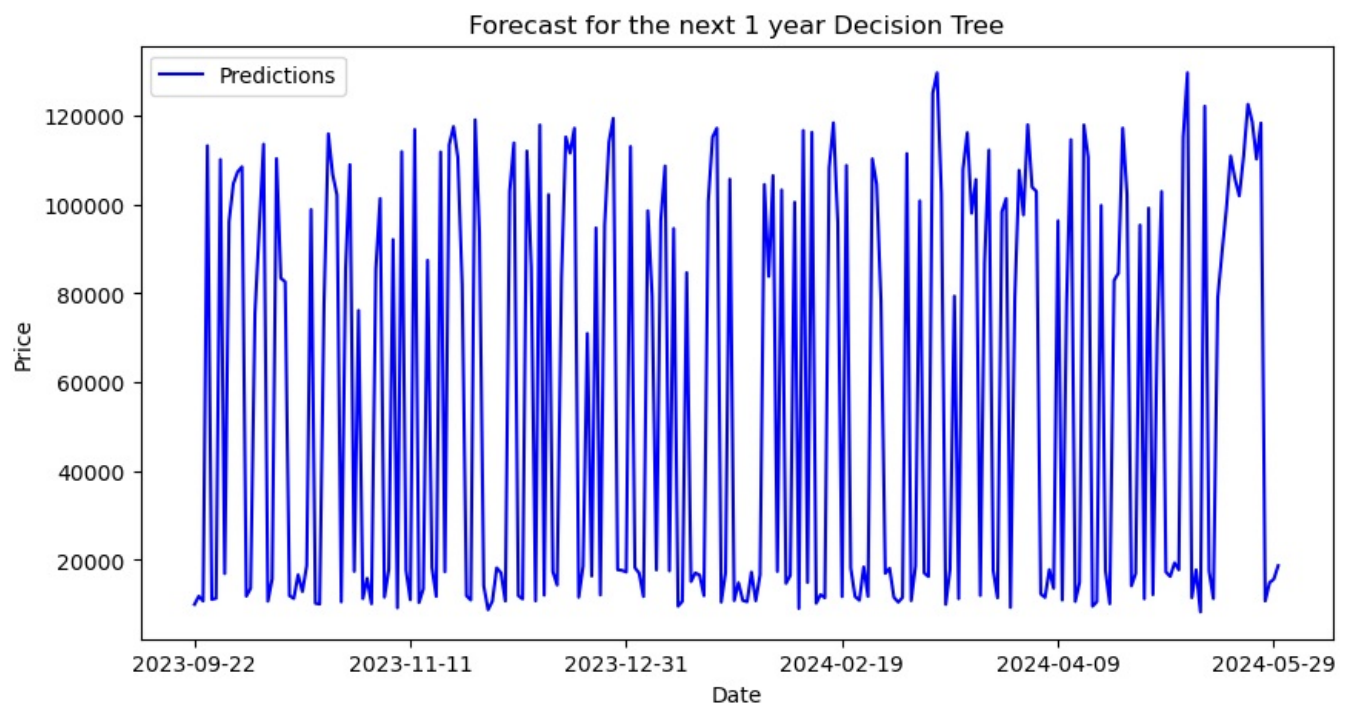
```
In [35]: print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_dt, predict), 4))
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_dt, predict), 4))
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_dt, predict)), 4))
print("(R^2) Score:", round(metrics.r2_score(y_test_dt, predict), 4))
print(f'Train Score : {model.score(x_train_dt, y_train_dt) * 100:.2f}% and Test Score : {model.score(x_test_dt,
errors = abs(predict - y_test_dt)
mape = 100 * (errors / y_test_dt)
accuracy = 100 - np.mean(mape)
print('Accuracy:', round(accuracy, 2), '%.')
```

Mean Absolute Error: 210.5259
Mean Squared Error: 16750369.3473
Root Mean Squared Error: 4092.7215
(R²) Score: 0.9919
Train Score : 100.00% and Test Score : 99.87% using Decision Tree Regressor.
Accuracy: 99.76 %.

```
In [36]: predictions = pd.DataFrame({"Predictions": predict})
predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_dt_NSE.csv")
#collects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_dt_NSE.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_dt_NSE.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_dt_NSE.csv")
```

```
In [37]: oneyear_df_pred = pd.read_csv("one-year-predictions_dt_NSE.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year Decision Tree", color=
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date
Predictions
Date
2024-05-12 8278.366536
Sell price and date
Predictions
Date
2024-03-12 129650.5
2024-05-09 129650.5



```
In [38]: onemonth_df_pred = pd.read_csv("one-month-predictions_dt_NSE.csv")
onemonth_df_pred.set_index("Date", inplace=True)
buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
print(onemonth_sell)
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month Decision Tree", colo
```

```
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date

Predictions

Date

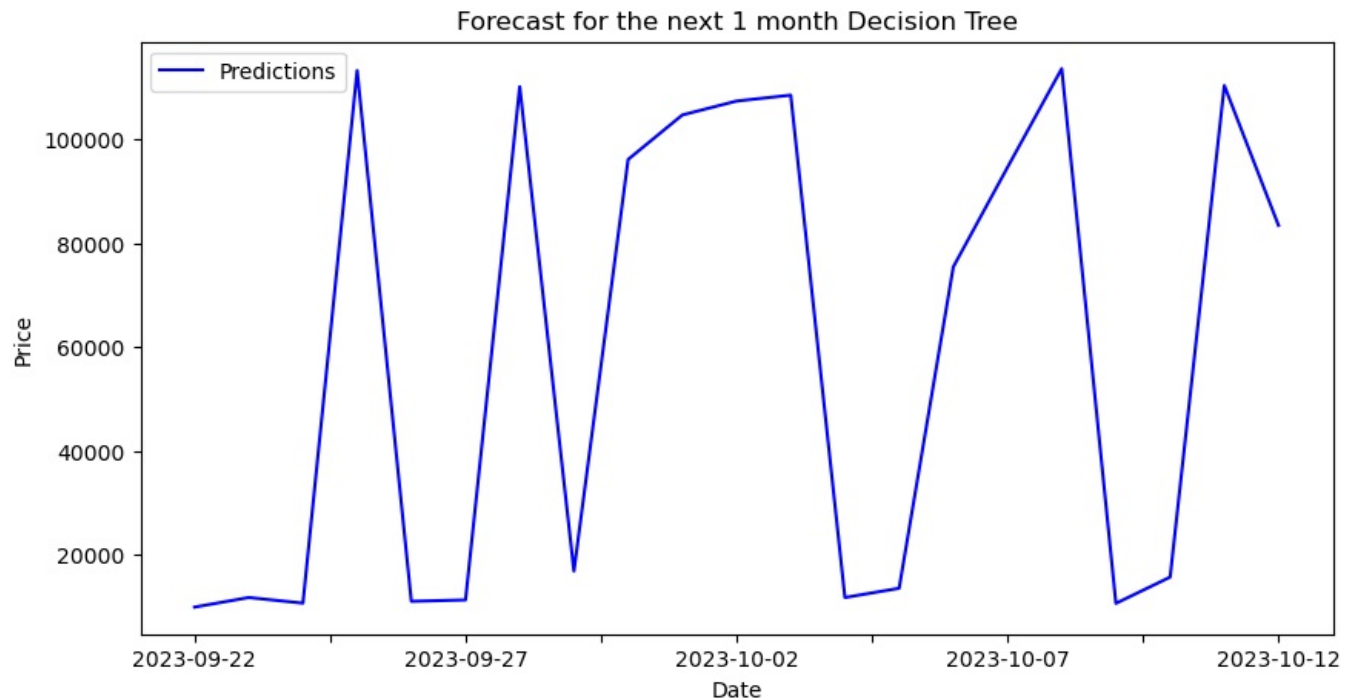
2023-09-22 9975.383464

Sell price and date

Predictions

Date

2023-10-08 113578.333333



```
In [39]: fivedays_df_pred = pd.read_csv("five-days-predictions_dt_NSE.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days Decision Tree", color=
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date

Predictions

Date

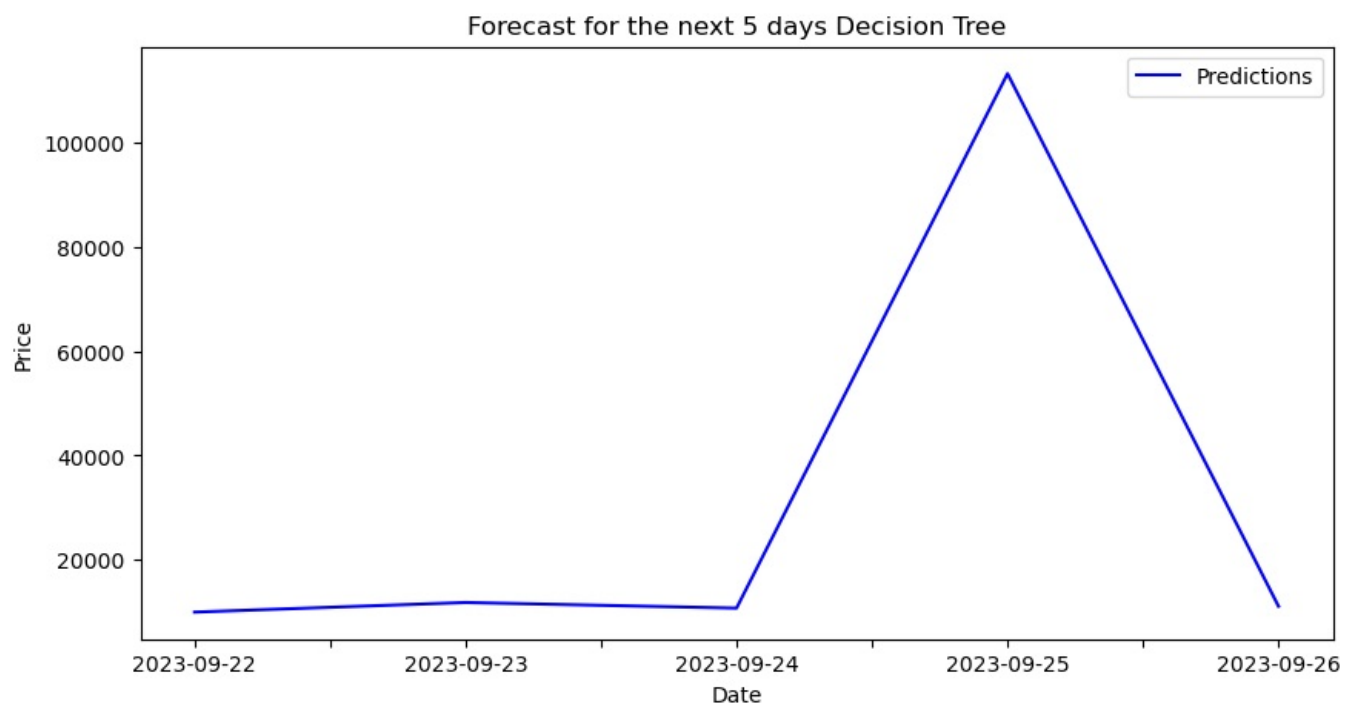
2023-09-22 9975.383464

Sell price and date

Predictions

Date

2023-09-25 113223.5



Logistic Regression - ValueError: Unknown label type: 'continuous'

Multilayer Perceptron

```
In [40]: temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values

In [41]: x_train_mlp, x_test_mlp, y_train_mlp, y_test_mlp = train_test_split(x, y, test_size=0.20, random_state=0)

In [42]: scale = StandardScaler()
x_train_mlp = scale.fit_transform(x_train_mlp)
x_test_mlp = scale.transform(x_test_mlp)

In [43]: # Define the parameter grid to search over
param_grid = {
    'hidden_layer_sizes': [(100,), (50, 50), (20, 20, 20)],
    'activation': ['relu', 'tanh'],
    'alpha': [0.0001, 0.001, 0.01],
    'learning_rate': ['constant', 'adaptive'],
}

# Create the MLP regressor
mlp = MLPRegressor(max_iter=1000)

rscv_mlp = RandomizedSearchCV(estimator=mlp, param_distributions=param_grid, cv=3, n_jobs=-1, verbose=2, n_iter=100)
rscv_fit_mlp = rscv_mlp.fit(x_train_mlp, y_train_mlp)

# Get the best parameters and best model
best_params = rscv_fit_mlp.best_params_
best_model = rscv_fit_mlp.best_estimator_

```

```
print(best_params)
print(best_model)
```

Fitting 3 folds for each of 10 candidates, totalling 30 fits

```
{'learning_rate': 'adaptive', 'hidden_layer_sizes': (20, 20, 20), 'alpha': 0.01, 'activation': 'relu'}
```

MLPRegressor(alpha=0.01, hidden_layer_sizes=(20, 20, 20),

learning_rate='adaptive', max_iter=1000)

C:\Users\LENOVO\anaconda3\Lib\site-packages\sklearn\neural_network\multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
warnings.warn(

```
In [44]: # Retrain the best model on the full training set
best_model.fit(x_train_mlp, y_train_mlp)

# Make predictions
predict = best_model.predict(x_test_mlp)
predict
```

C:\Users\LENOVO\anaconda3\Lib\site-packages\sklearn\neural_network\multilayer_perceptron.py:691: ConvergenceWarning: Stochastic Optimizer: Maximum iterations (1000) reached and the optimization hasn't converged yet.
warnings.warn(

```
Out[44]: array([ 9750.57085981, 12274.56749219, 11515.46254283, 112852.04951011,
11773.80451217, 11926.39177218, 109995.53167295, 16392.61294833,
96122.04171906, 104752.58811451, 107110.73054 , 108383.12414854,
11875.62801637, 13519.92491303, 75278.22712683, 94705.22589006,
113465.46893065, 11490.00131807, 15611.54036885, 110268.63336145,
83732.76162836, 82965.36262606, 11876.75815013, 11166.16784695,
16245.31877927, 12525.30816961, 18325.28757749, 98913.17167938,
10641.03933647, 10080.03252419, 77424.52725041, 115669.84287982,
106893.89433519, 102583.16355501, 11353.23141648, 85453.45765914,
108769.24322959, 17119.44219182, 76438.19126259, 11929.10308083,
16040.90987645, 11021.1752282 , 85544.30920869, 101359.47314812,
12156.19104549, 17338.17846386, 92230.30402828, 9436.07033712,
111832.90277091, 17287.58494199, 11773.58028097, 116632.51954107,
11147.52069001, 13409.63650613, 87312.30721108, 17569.91438126,
12002.54379252, 111681.1495223 , 16461.05911033, 113341.41842171,
117309.50413626, 110680.28117745, 81778.3685585 , 12222.56562794,
11589.49830048, 118980.69603111, 93652.05304386, 14024.84322958,
8943.69862705, 11374.55769778, 17521.3790637 , 16680.7424742 ,
11373.01614506, 102880.19998789, 113641.55418719, 12216.14847827,
11626.91055282, 112205.84600638, 85880.60663495, 11432.8477519 ,
117909.6813813 , 12350.84509778, 102348.27669019, 16919.65973503,
13884.87525569, 83448.30001776, 115399.77421177, 111478.56654929,
117100.00670198, 12094.06161236, 18223.86189011, 70749.71776816,
16056.00369913, 94897.19868587, 11949.09286648, 95470.48248656,
113750.22661478, 119099.02606923, 17280.54587634, 17314.90764987,
16969.72390511, 113069.43496035, 17915.66411124, 16504.48183913,
12083.65112112, 98627.28446812, 79533.84839215, 17311.06819204,
96295.6070008 , 108467.76919199, 17160.51073473, 94839.67434716,
9596.75666166, 11339.40362487, 84656.65154902, 14770.92959951,
16645.65644999, 15877.27429859, 12135.52442643, 100555.74668472,
115142.26747012, 116877.00755834, 11279.74151071, 16543.47123416,
105799.09270051, 11243.91047343, 14488.37297891, 11684.37506237,
11204.95463501, 16988.99974064, 11480.11118158, 15974.72031306,
104536.0952518 , 83934.02647632, 106408.44364732, 16838.90085909,
103184.78170841, 14556.00369466, 15923.5525199 , 100404.35144115,
9517.64397479, 116624.56909364, 14622.04081688, 115945.00775635,
10824.29689952, 13016.45618562, 11934.12967821, 107868.5831405 ,
118170.20966182, 95850.18878098, 11780.25055814, 108798.21077611,
17584.90374763, 12242.51221509, 11467.30241432, 17971.67677335,
12242.11162062, 110130.02786151, 104229.73712707, 78146.23358816,
16467.34640662, 17744.30010019, 12204.27641928, 10516.04296935,
12043.41897983, 111418.372361 , 11501.37514988, 18367.7792988 ,
100763.68294751, 16748.52093933, 15975.44600871, 124776.10758901,
129234.67519828, 102222.57172724, 10193.81799571, 17547.62621602,
80433.71682982, 11458.70272603, 108042.72774154, 115740.10597185,
97831.87873577, 105502.40758052, 12186.40184054, 86911.05208245,
112028.61583479, 16987.58648822, 12046.96648609, 98254.49486681,
100924.14570464, 9244.61494033, 79131.52217432, 107736.73736567,
97681.61149382, 117652.12559716, 104094.19748031, 103189.05161087,
12617.07946669, 11595.81232948, 17451.86116099, 13110.11220899,
96073.54931359, 11536.74817459, 78193.79088789, 115056.59593631,
11240.45123258, 14488.9777658 , 117759.01312501, 110515.85594664,
9538.67412741, 11378.31613054, 99708.52200009, 16949.49786404,
10991.4525677 , 82670.31364994, 84625.39330683, 117024.39595626,
102369.13598797, 14143.98903584, 16345.70900986, 95287.85193264,
11573.36872593, 99412.58378706, 12545.92300598, 71890.29018418,
102838.54929333, 16566.23475391, 15548.22819436, 18766.64445487,
16833.99995904, 115172.91426111, 129885.7499594 , 12002.42028085,
17232.35800507, 8759.33106847, 121797.22977714, 17133.62177556,
9838.82610834, 78918.6799895 , 89441.31416836, 98736.69049042,
111206.28540573, 105754.20027468, 101610.96060094, 110561.01481839,
122183.02346625, 118542.03007514, 110356.25034522, 118305.47298643,
```

11573.45244753,	14251.12806064,	15002.54072475,	18179.73213047,
11603.24387056,	117685.05012242,	17036.34093416,	91823.26717001,
11308.6448133 ,	17251.08798012,	12831.65664201,	16746.79588497,
104142.81691848,	99176.31176617,	12355.72151389,	16217.01614273,
83788.08893844,	117005.94005254,	15762.99474478,	103511.97779526,
81618.81756404,	78852.96551555,	95885.78568559,	104002.90582755,
80117.84275389,	10863.99637107,	248639.04846854,	17880.25763647,
84365.15091896,	9993.47412431,	112001.05390138,	16167.73514463,
86840.01405635,	16865.38647402,	17613.30588898,	11716.60116063,
16128.13827619,	101272.77806563,	95385.84805896,	103016.84212627,
88639.19136179,	11631.63086406,	16754.78319068,	119024.83036131,
10989.58109763,	108179.18948218,	109992.18194999,	15189.93130359,
11380.38083971,	16805.77114851,	100190.82257561,	115720.75413274,
11845.57950547,	109926.85132581,	118638.52008617,	18800.28291197,
84209.61505564,	101021.7675335 ,	11222.75224626,	15251.09297302,
106691.51309685,	96407.82651863,	116189.12625727,	15796.59304394,
113891.2273579 ,	12027.02840552,	12529.59466697,	119277.14910298,
17385.96781615,	113346.35755769,	11704.57099049,	100431.7688222 ,
12166.1129607 ,	103696.14489029,	16609.17492008,	118692.2054152 ,
15594.22017489,	16922.12974321,	18589.85519467,	114583.20092837,
17891.15371262,	15273.45432204,	11632.96736057,	11231.83141179,
11486.15544051,	100441.1318528 ,	108587.4918283 ,	16360.06044385,
85569.61373461,	17254.26022151,	101692.42255217,	18802.75743491,
12581.85769763,	103200.65498483,	112364.4917187 ,	16733.08004904,
114899.05633665,	116939.1895838 ,	16857.28337719,	104620.74471664,
17407.32682502,	85735.93034315,	106902.90463785,	104106.92202437,
12020.32847319,	127031.36165303,	12431.56130413,	11458.20029469,
117768.37768875,	78043.08837778,	100790.49364805,	14423.69838331,
17176.36223586,	99081.00505739,	12258.22866659,	11569.33471592,
109834.75070866,	9885.05517228,	117570.48665125,	11573.8597279 ,
109513.0779866 ,	15664.28326261,	95652.00339579,	107031.19678064,
16919.18471917,	15391.37341952,	11728.71747158,	16847.06181585,
79297.29183336,	14137.45475306,	12322.51990555,	12704.94899327,
85714.21803326,	17719.43940285,	15646.48401237,	12534.63032736,
107434.13622062,	11735.03886442,	104748.96860653,	76408.80282772,
84848.63423086,	85596.41394943,	109627.70860664,	16749.08252709,
14967.79750608,	106189.41968004,	109046.99551486,	11828.95373503,
112444.20699161,	12020.26726764,	121782.78509247,	11485.23055484,
76088.91010738,	100438.21357376,	85488.0843568 ,	110493.75293347,
72812.49256232,	116923.09181022,	16859.2598435 ,	12391.96078479,
16948.2745747 ,	112915.75141825,	102185.52797313,	75559.89853311,
11434.63825026,	97813.87560398,	15716.60688306,	97055.35976454,
9162.37670239,	15726.85306704,	11609.50671861,	96297.63321443,
16921.16498068,	10956.76484508,	101327.43242404,	116246.14490144,
17416.3656254 ,	11001.2862256 ,	107511.39204485,	17842.57735287,
114915.4101661 ,	18907.16795753,	11286.26251283,	109015.77603342,
11670.93473226,	96083.93928054,	119710.64152288,	11173.69285662,
111175.34690441,	17678.58984185,	12190.13532894,	107739.24092355,
101138.07990427,	9985.48807037,	104726.0337146 ,	120564.36246215,
113204.49876195,	9268.61031707,	120606.95486749,	104335.37096986,
11883.1757244 ,	17849.38609142,	15677.27074326,	118899.42283957,
12242.82946513,	12254.13206521,	11152.41058987,	104212.81015377,
15975.85516881,	77434.48834146,	110232.64857058,	11238.03643157,
10980.56800565,	101360.77411531,	94780.7357612 ,	112107.60486082,
122780.45397989,	76635.33850237,	14173.31702225,	11536.39581027,
17697.94680842,	101236.44240719,	106556.85388642,	108066.11401862,
11146.11962736,	94109.9479449 ,	103624.00450972,	17460.21852447,
9390.88447225,	13834.45417061,	127509.42794288,	12191.64722122,
108390.80217999,	116129.84937285,	79703.99640785,	18626.89845361,
17268.81435484,	10991.8250923 ,	118933.65137468,	85458.09661692,
83152.26976926,	83449.34620446,	95438.38468052,	17204.29671585,
16824.0338049,	75212.70938342,	107027.40655429,	10880.30329887,
11577.00596608,	16597.95837425,	104784.0993625 ,	93566.74484688,
12168.10378929,	130024.98842412,	17023.05350825,	12420.1875527 ,
11503.09333356,	108701.38839041,	9944.40020435,	15085.29580883,
101122.66402423,	17889.5797374 ,	15967.3103454 ,	97899.65042418,
114110.37855036,	113785.57939832,	103173.66633287,	16482.96971787,
18997.68882973,	16893.58164956,	79923.41295886,	11704.04685454,
13817.82144579,	11055.18378261,	11788.42041662,	74717.49496324,
120481.10375205,	12574.70280418,	12075.2811824 ,	12301.3624063 ,
115183.36612027,	120094.52943086,	121486.71958417,	83005.3427854 ,
103098.95736659,	107034.45919414,	85465.55316593,	125545.6714407 ,
82019.62210764,	14373.96156656,	11187.64055179,	84453.74519935,
17079.15375755,	8920.35490632,	109126.38907625,	12686.32834367,
16720.36537747,	11999.39564887,	17743.06642247,	17343.08602232,
117151.73520154,	114748.48662292,	17393.79595545,	113469.79796909,
93656.91733132,	11508.72491515,	112053.23196636,	11304.75421413,
101870.27049819,	97943.44276396,	104698.18115958,	98325.13117748,
103968.53784398,	9775.27082575,	11489.55712198}}	

```
In [45]: print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_mlp, predict), 4))
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_mlp, predict), 4))
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_mlp, predict)), 4))
```



```

print("(R^2) Score:", round(metrics.r2_score(y_test_mlp, predict), 4))
print(f'Train Score : {model.score(x_train_mlp, y_train_mlp) * 100:.2f}% and Test Score : {model.score(x_test_m',
errors = abs(predict - y_test_mlp)
mape = 100 * (errors / y_test_mlp)
accuracy = 100 - np.mean(mape)
print('Accuracy:', round(accuracy, 2), '%.')

```

Mean Absolute Error: 532.9014

Mean Squared Error: 31390889.5894

Root Mean Squared Error: 5602.7573

(R^2) Score: 0.9848

Train Score : 100.00% and Test Score : 99.87% using Multilayer Perceptron Regressor.

Accuracy: 98.07 %.

```

In [46]: predictions = pd.DataFrame({"Predictions": predict})
predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_mlp_NSE.csv")
#collects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_mlp_NSE.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_mlp_NSE.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_mlp_NSE.csv")

```

```

In [47]: oneyear_df_pred = pd.read_csv("one-year-predictions_mlp_NSE.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year Multilayer Perceptron")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()

```

Buy price and date

Predictions

Date

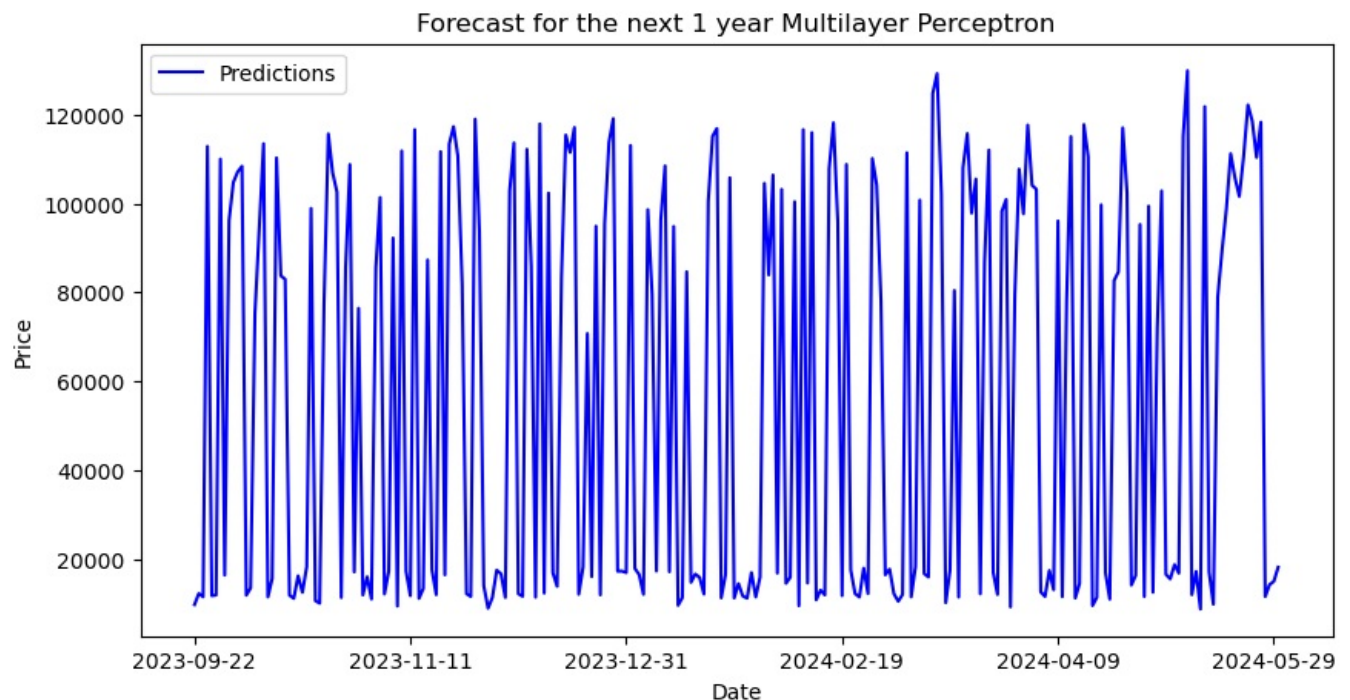
2024-05-12 8759.331068

Sell price and date

Predictions

Date

2024-05-09 129885.749959



```

In [48]: onemonth_df_pred = pd.read_csv("one-month-predictions_mlp_NSE.csv")
onemonth_df_pred.set_index("Date", inplace=True)
buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]

```



```
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
print(onemonth_sell)
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month Multilayer Perceptron")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date

Predictions

Date

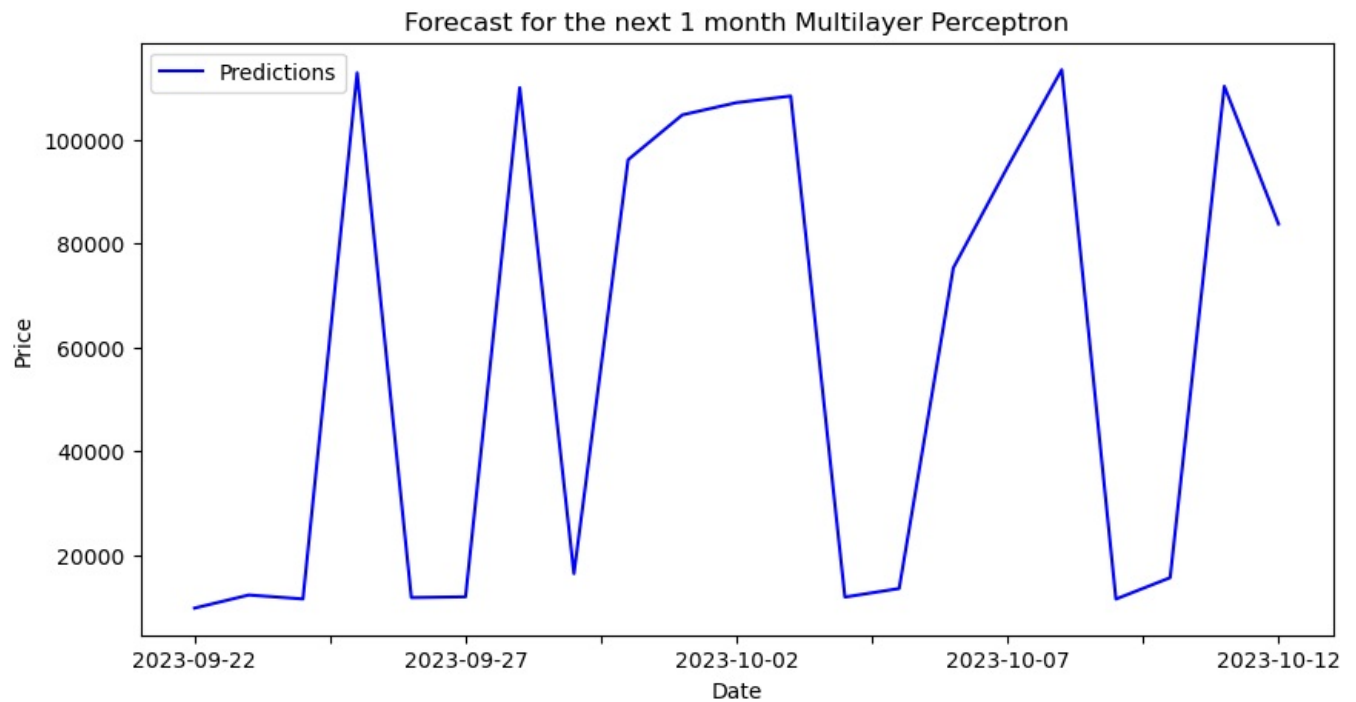
2023-09-22 9750.57086

Sell price and date

Predictions

Date

2023-10-08 113465.468931



```
In [49]: fivedays_df_pred = pd.read_csv("five-days-predictions_mlp_NSE.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days Multilayer Perceptron")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date

Predictions

Date

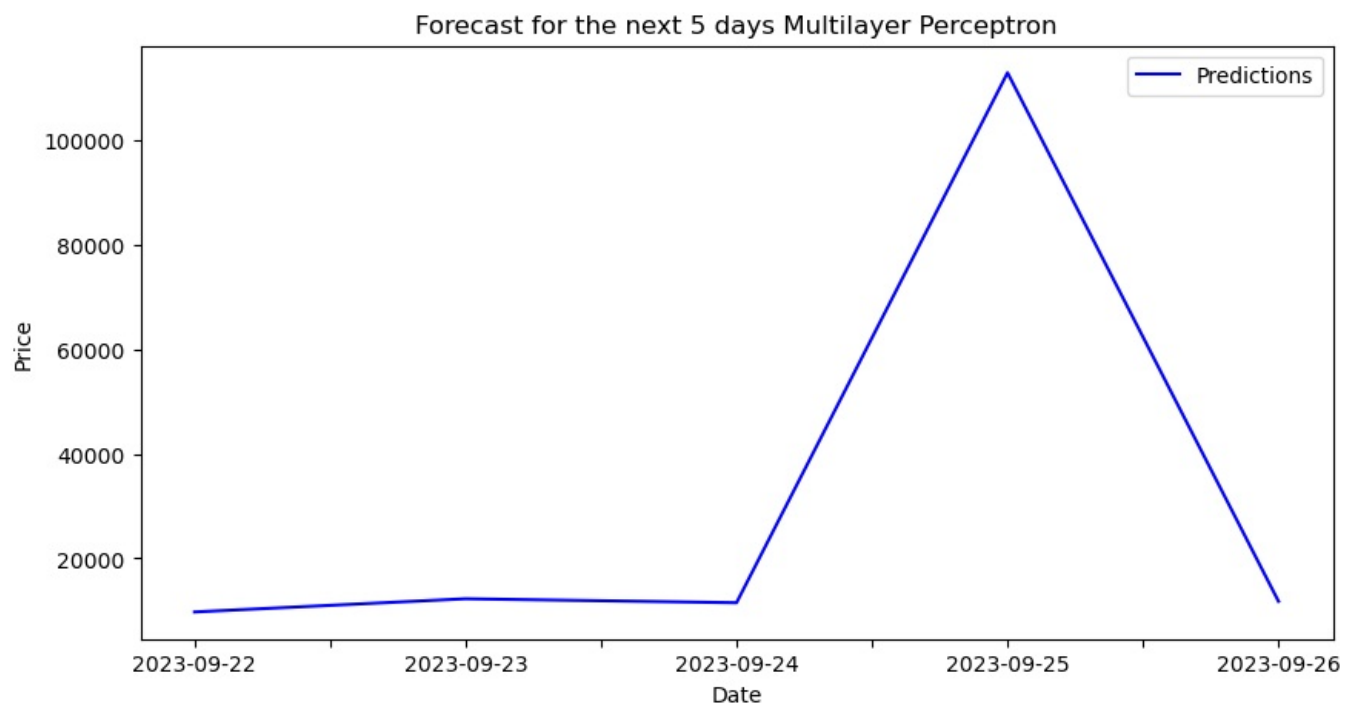
2023-09-22 9750.57086

Sell price and date

Predictions

Date

2023-09-25 112852.04951



KNN

```
In [50]: temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values

In [51]: x_train_knn, x_test_knn, y_train_knn, y_test_knn = train_test_split(x, y, test_size=0.20, random_state=0)

In [52]: scale = StandardScaler()
x_train_knn = scale.fit_transform(x_train_knn)
x_test_knn = scale.transform(x_test_knn)

In [53]: # Define the parameter grid to search over
param_grid = {
    'n_neighbors': range(1, 21), # Number of neighbors to consider
    'weights': ['uniform', 'distance'], # Weighting scheme for neighbors
    'p': [1, 2], # Power parameter for the Minkowski distance metric
}

# Create the KNN classifier
knn = KNeighborsRegressor(n_neighbors=5) # Set the number of neighbors (k) as 5

# Initialize the RandomizedSearchCV object
rscv_knn = RandomizedSearchCV(estimator=knn, param_distributions=param_grid, cv=5, n_jobs=-1, verbose=2, n_iter=50)

# Perform hyperparameter tuning
rscv_fit_knn = rscv_knn.fit(x_train_knn, y_train_knn)

# Get the best parameters and best model
best_params = rscv_fit_knn.best_params_
best_model = rscv_fit_knn.best_estimator_

print(best_params)
print(best_model)
```

Fitting 5 folds for each of 50 candidates, totalling 250 fits
 {'weights': 'distance', 'p': 1, 'n_neighbors': 7}
 KNeighborsRegressor(n_neighbors=7, p=1, weights='distance')

```
In [54]: # Retrain the best model on the full training set
best_model.fit(x_train_knn, y_train_knn)

# Make predictions
predict = best_model.predict(x_test_knn)
predict
```

```
Out[54]: array([[ 9605.12463747, 11826.64185141, 10809.76739483, 114113.06710317,
10953.91971578, 11392.81967568, 108757.04088069, 16992.65325406,
96624.08463328, 103233.43243976, 105247.58598494, 107732.83421923,
11825.4879304 , 13596.1811198 , 75973.7255997 , 96048.41950675,
113242.87551048, 10735.93657236, 15744.65039999, 108560.61864594,
84924.17416881, 84471.7936945 , 11767.14984919, 11112.92160192,
16728.24521578, 12649.82924981, 18703.21860514, 99829.75297922,
10391.25308553, 9686.11974192, 76972.52569228, 114409.95609694,
107321.82491769, 102222.88654244, 10665.75422943, 86052.13062854,
110532.40254117, 17422.79063953, 77359.65338818, 11232.46860705,
15825.49106493, 10196.78287871, 86247.49570902, 101827.83474851,
11517.2787243 , 17745.98133217, 94846.72036412, 9171.11420607,
110093.23440627, 17379.34010708, 10872.79182956, 114617.40469367,
10573.16981487, 13548.08836755, 87058.90948342, 17958.19005703,
11823.43884206, 111448.73631298, 17338.12761879, 111935.73269068,
116304.22183672, 109099.81797109, 81627.31361219, 11870.57007745,
10935.67719348, 118963.97873693, 94741.12493898, 14301.90705605,
9034.61299744, 10791.01142064, 18260.14284351, 17048.16037925,
10752.41334237, 104373.64300566, 114837.4610842 , 11803.48878512,
11195.52269874, 110965.40238185, 85320.35872599, 10745.15795192,
117878.16627076, 12056.33581133, 101762.92429223, 17208.57245402,
14414.12353345, 84483.587829 , 114877.71392901, 112037.12960642,
116343.42016252, 11557.36292638, 18662.14392428, 72491.22249621,
16356.85024722, 96177.94469222, 11981.61918467, 96890.73637153,
114944.95630996, 118947.73760116, 17969.45634498, 17796.6465816 ,
17350.40972862, 112342.85443182, 18088.47132572, 17099.64033207,
11673.98591064, 99615.73254154, 78084.81701428, 17616.2481078 ,
96583.11749928, 106647.21774485, 17579.93378656, 96897.88226505,
9200.49632533, 10704.80868129, 85129.0651705 , 15031.42447037,
17178.65959147, 16564.20835429, 11940.12775876, 99162.79377075,
114756.31137546, 118504.07233642, 10603.35041843, 17010.75678286,
106865.17238044, 10870.81125384, 14780.00124376, 10805.11869708,
10527.3206531 , 17275.8327415 , 10794.39871988, 16658.85392091,
103798.92865502, 83146.32486309, 104855.93623527, 17402.97681592,
103584.16063282, 14793.32889308, 16469.52519903, 100764.56977026,
9136.34498283, 117352.79458922, 14804.3446624 , 112806.06221281,
10265.03407321, 12043.32602965, 11346.17495302, 107749.62334324,
118521.96632665, 96994.59845773, 11794.92433437, 109595.42871516,
17980.31980931, 11624.5634093 , 10824.93232403, 18333.05322111,
11744.77214207, 107504.95620247, 105252.53311298, 76913.16595314,
16972.44307088, 18072.10217219, 11600.30463828, 10454.13507628,
11457.14820363, 110154.29304034, 10854.51274771, 18687.83342921,
101240.41962222, 17254.36426323, 16191.95989378, 119936.28318875,
126508.63392781, 102148.39742298, 9959.00713627, 17949.8932755 ,
78525.54367785, 11268.15101678, 107075.5385881 , 112294.52554486,
103586.36272124, 104602.92283309, 11992.18437776, 85341.51655092,
111985.07684865, 17360.71779463, 11493.33779511, 99306.3978278 ,
102198.45120443, 9368.88367224, 80477.32321178, 107605.9063147 ,
97253.51841952, 115370.38814146, 105003.56492084, 102887.74722118,
12223.27135948, 11520.4670297 , 17843.26811617, 13372.91479181,
95353.57597441, 10835.527944 , 77669.89036334, 113955.89367898,
10747.77638104, 14845.50704873, 117428.14181054, 109532.23272219,
9406.92090623, 10543.08305648, 99219.03661942, 17725.80653012,
10235.21771435, 84227.7434483 , 85699.04046971, 118750.0905749 ,
100212.27234547, 13862.48043677, 16917.49154341, 94580.32999077,
11136.54864206, 98644.38045407, 12154.37602902, 73357.11796634,
101754.30990089, 17161.75200958, 16605.80079593, 19387.81269476,
17556.8608218 , 115487.28570409, 129123.16545457, 11530.44197205,
17717.93658845, 8687.76675154, 121182.5571612 , 17537.86204037,
10521.32065908, 78730.3049352 , 87690.10066926, 99100.72704321,
110430.28531596, 104819.91676871, 104641.62245964, 110107.50831586,
121647.26201479, 118379.48782156, 110645.78874475, 117628.03401611,
10746.83290138, 14787.66860612, 15470.75158071, 18599.59435125,
10850.69387467, 117100.82735434, 17321.99398624, 92705.69883652,
10813.73940644, 17997.10015821, 13171.72991145, 17173.86226602,
104724.06153175, 97696.76824295, 12071.62819712, 16176.7179259 ,
84945.29204687, 116603.88432496, 16916.35708453, 104352.47797828,
78278.31426405, 78883.61482608, 98416.97176688, 103848.17227731,
79206.34069631, 10377.93439724, 111354.37407714, 18287.12135269,
84555.50178689, 9978.59054036, 110858.58906987, 16591.16661214,
82622.75774903, 17300.05748079, 18031.83978665, 11335.05787548,
16838.34807231, 102136.17813147, 95879.86225346, 102860.9152615 ,
85845.46452745, 11382.67638917, 17190.59984883, 118766.13876877,
10832.19562645, 108249.37417958, 108260.52680741, 15730.13634072,
11446.17466054, 17537.26334174, 101747.93134273, 115780.38211462,
11480.99010952, 111356.40050634, 118462.51319118, 19466.2308474 ,
```

85698.47347515, 99973.02342088, 10984.93535837, 15173.7077309 ,
106702.97824203, 97942.53880509, 115023.68705663, 15960.28108617,
114372.27893556, 11911.06160533, 12177.54457888, 119125.30400013,
17744.28507088, 113573.00398209, 10821.30627983, 99315.66325238,
11661.95441062, 104531.23402352, 17100.39161064, 116650.64023182,
15906.15611042, 17453.53375644, 19581.63625862, 115124.37483888,
18320.30949755, 15786.08986772, 11300.24615744, 10471.27640349,
11479.70611124, 99607.42598612, 110477.99916488, 16941.65258104,
83500.65357469, 17797.48489595, 101161.84325222, 19426.58060877,
12081.34707581, 102949.61561928, 112144.36515984, 17627.81164955,
116291.32409699, 115558.49322813, 17533.24866593, 105125.39318982,
17846.11762893, 84786.34695635, 106537.73609097, 104324.32035977,
12131.14529178, 127342.01378568, 11842.61404067, 10847.21159057,
119200.24188999, 79157.93895632, 101034.38326179, 14827.7658547 ,
17555.9835649 , 97961.15692587, 11689.26703724, 11230.9969235 ,
110174.80788543, 9175.86379748, 118077.29097356, 11388.56968306,
110524.2085321 , 15736.51510194, 95861.09591763, 109307.40490325,
17421.98321617, 16211.37658905, 11135.98900235, 17498.8226958 ,
79922.18046645, 14514.09817592, 12173.68350469, 12902.48076327,
82411.36328163, 18206.21310559, 15771.50216754, 11849.69531722,
106780.58132164, 11644.06648162, 104681.203461 , 77068.19286841,
85314.06420659, 85399.11171869, 111043.6513169 , 17309.82341816,
15080.81295109, 106520.60075889, 108171.06185015, 11588.86412519,
112025.44552666, 11412.0188335 , 121627.17493352, 10743.52697381,
77905.24818497, 99658.85233458, 85906.25550012, 110867.0707095 ,
78626.99295392, 114152.50805516, 17338.1761276 , 12082.25436155,
17804.62020397, 112915.39188237, 102336.36196629, 75333.56051138,
11742.09843792, 99244.02377201, 15795.21951 , 97294.71704582,
9145.35467021, 15832.59523545, 11072.37853653, 96042.79934081,
17441.35348775, 10967.50071029, 101497.93536282, 114653.13668092,
18029.22633699, 10725.97780874, 107305.35487289, 17967.67105633,
115621.93204294, 19419.04275335, 10928.11873262, 109560.11469157,
10872.79400015, 97044.9636009 , 119963.47748759, 10616.25816579,
111349.2781388 , 18119.18213254, 11495.54266382, 107443.40195619,
104563.08903067, 11019.65477914, 104761.78538096, 119586.18988221,
112064.81032157, 9170.48037454, 119010.64880354, 104007.8567376 ,
11002.80018368, 18311.48694014, 16046.85064713, 116281.06297846,
12124.00824628, 11576.13587702, 11108.94942521, 105617.97314866,
16315.41147501, 77001.8928084 , 110336.06818849, 10841.16651974,
10759.84706726, 101541.30985473, 94750.35462219, 111399.19813915,
122234.18180632, 79805.91470196, 14557.56144866, 10839.13237917,
17834.48169826, 102250.5026744 , 106945.58924471, 107598.40682357,
10522.15615314, 95140.04326677, 103613.69701022, 17878.03305127,
9389.2220429 , 14462.06755106, 125993.18516166, 11845.05265903,
108389.58981312, 117317.52939542, 78499.37531659, 19593.25609149,
17773.72601219, 10404.25031717, 118386.52901615, 85633.78660388,
84079.47773788, 83885.45507166, 96652.95516642, 17325.87103591,
17702.20815108, 77551.26595843, 108734.25504069, 10870.20698818,
11283.12518147, 17073.35107314, 104580.0366737 , 95838.96725324,
12271.81613301, 128183.93341089, 17595.56943043, 12246.64477952,
10400.94415186, 108701.19354979, 9684.35450112, 15382.44883687,
101042.62564455, 18340.63596255, 16033.93176789, 96257.09473895,
115163.58361961, 113563.1347581 , 106255.39067295, 17061.43276005,
19505.63102669, 17404.0619324 , 79705.67758269, 11308.83149535,
14600.64703582, 10366.40057975, 11890.12745331, 77351.49729665,
119637.64039404, 12462.03522257, 11795.20620674, 12129.37936449,
116265.72935024, 118960.62163618, 119648.10386279, 84773.63886386,
105124.82511157, 106732.46440953, 85453.93191566, 126034.75271597,
81012.46059904, 14632.24299387, 10430.51025807, 85668.30152148,
17678.50101355, 8761.55397296, 109480.03501771, 12971.64608743,
17236.20000245, 11407.07153872, 18106.81411729, 17643.11054016,
118358.34654031, 115304.41723365, 17674.31160445, 112687.09671175,
94883.06417929, 11454.37311079, 111575.76045414, 10733.19383957,
102899.76147724, 98650.92501148, 101706.95052424, 98461.0985497 ,
103338.51082342, 9489.52362414, 11401.2447548])

```
In [55]: print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_knn, predict), 4))
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_knn, predict), 4))
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_knn, predict)), 4))
print("(R^2) Score:", round(metrics.r2_score(y_test_knn, predict), 4))
print(f'Train Score : {model.score(x_train_knn, y_train_knn) * 100:.2f}% and Test Score : {model.score(x_test_knn, y_test_knn) * 100:.2f}%')
errors = abs(predict - y_test_knn)
mape = 100 * (errors / y_test_knn)
accuracy = 100 - np.mean(mape)
print('Accuracy:', round(accuracy, 2), '%.')
```

Mean Absolute Error: 579.8314
Mean Squared Error: 1035357.1193
Root Mean Squared Error: 1017.525
(R^2) Score: 0.9995
Train Score : 100.00% and Test Score : 99.87% using KNN Regressor.
Accuracy: 99.08 %.

```
In [56]: predictions = pd.DataFrame({"Predictions": predict})
```

```

predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_knn_NSE.csv")
#collects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_knn_NSE.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_knn_NSE.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_knn_NSE.csv")

```

```

In [57]: oneyear_df_pred = pd.read_csv("one-year-predictions_knn_NSE.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year KNN", color="blue")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()

```

Buy price and date

Predictions

Date

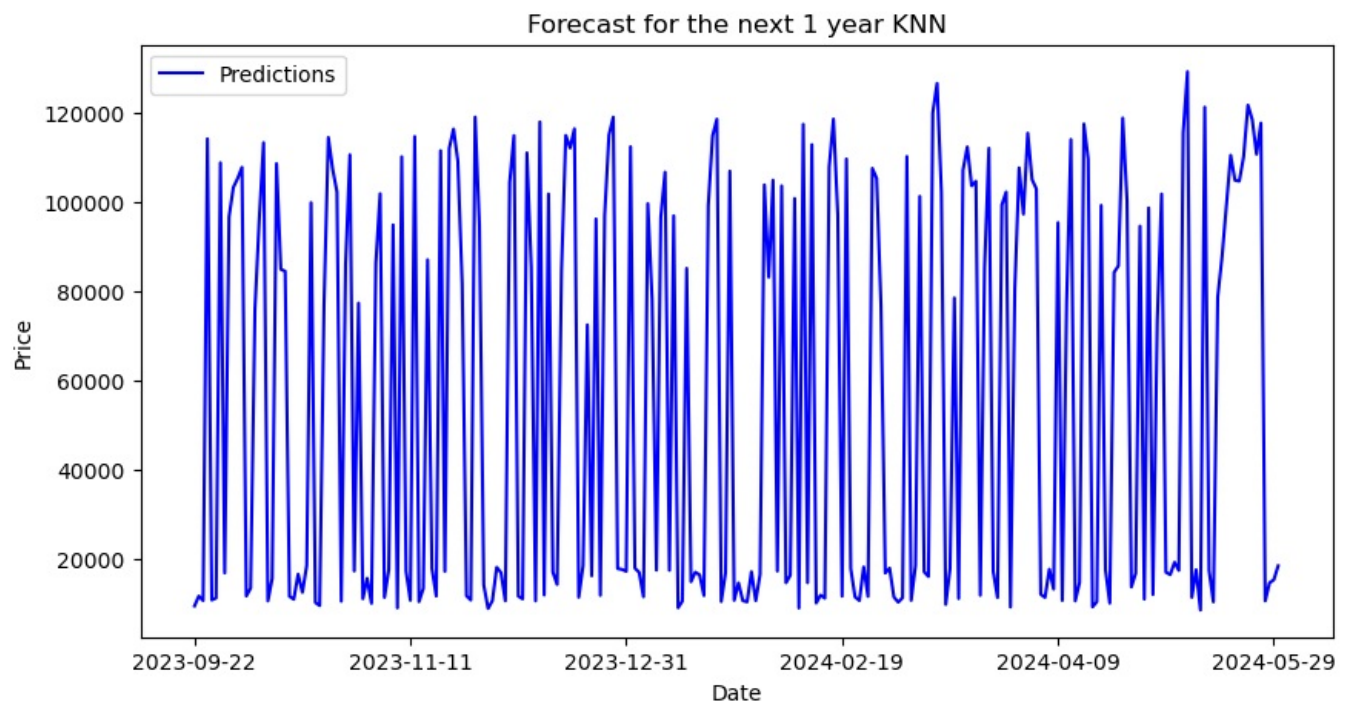
2024-05-12 8687.766752

Sell price and date

Predictions

Date

2024-05-09 129123.165455



```

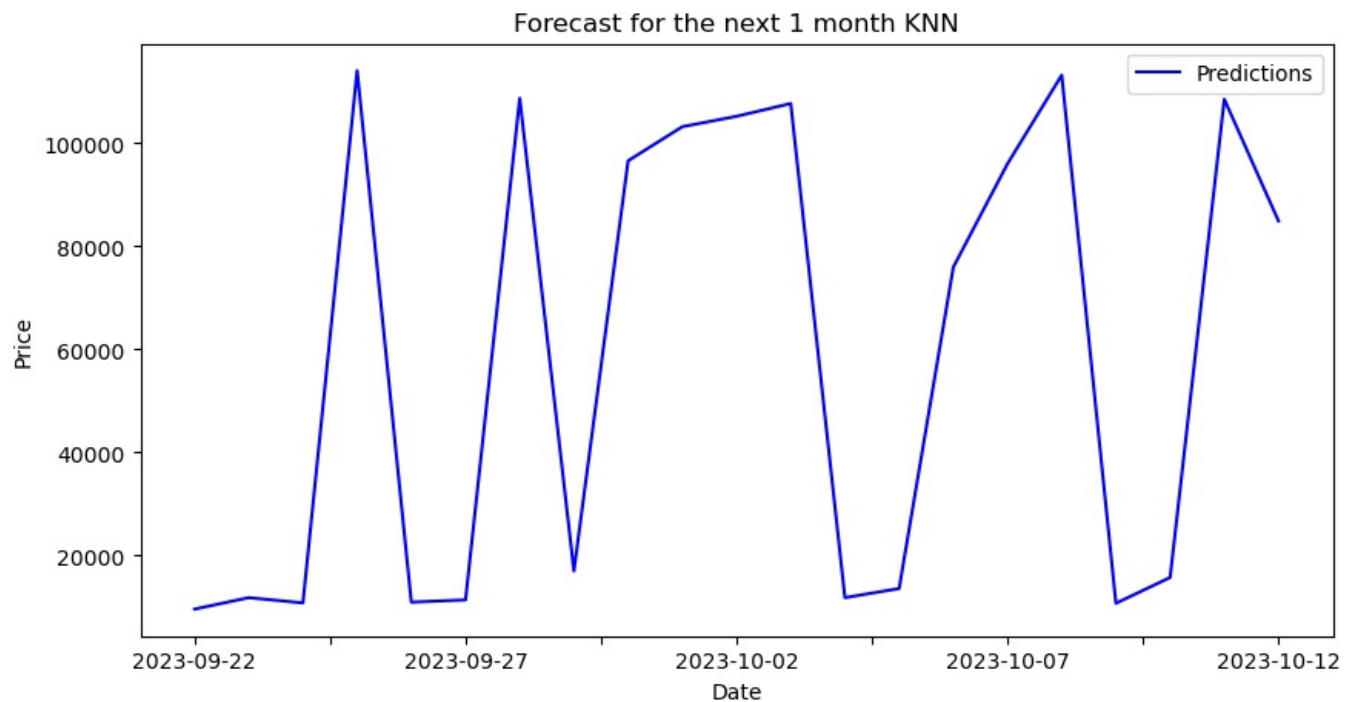
In [58]: onemonth_df_pred = pd.read_csv("one-month-predictions_knn_NSE.csv")
onemonth_df_pred.set_index("Date", inplace=True)
buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
print(onemonth_sell)
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month KNN", color="blue")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()

```

```

Buy price and date
      Predictions
Date
2023-09-22  9605.124637
Sell price and date
      Predictions
Date
2023-09-25  114113.067103

```



```

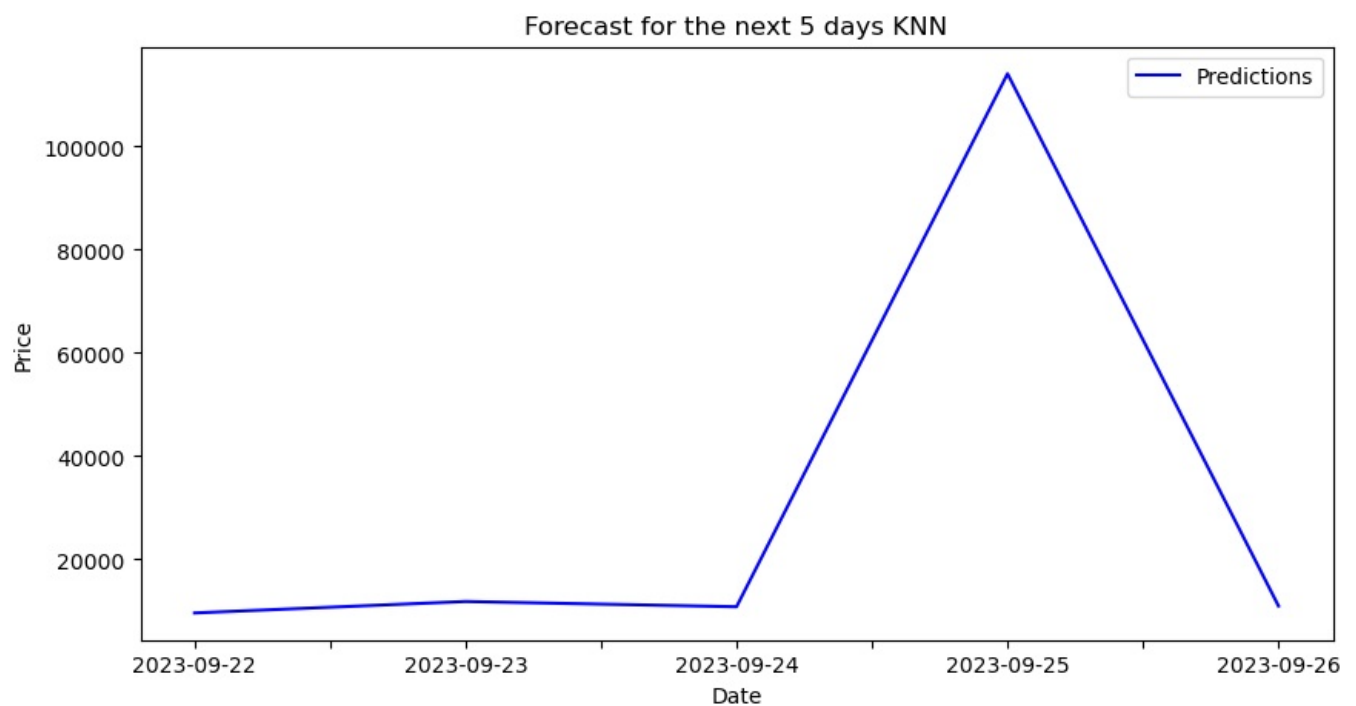
In [59]: fivedays_df_pred = pd.read_csv("five-days-predictions_knn_NSE.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days KNN", color="blue")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()

```

```

Buy price and date
      Predictions
Date
2023-09-22  9605.124637
Sell price and date
      Predictions
Date
2023-09-25  114113.067103

```



Clustering

```
In [60]: temp = nse_data
temp = temp.drop('Adj Close', axis=1)
x = temp.iloc[:, 0:12].values
y = nse_data.iloc[:, 4].values

In [61]: x_train_cls, x_test_cls, y_train_cls, y_test_cls = train_test_split(x, y, test_size=0.20, random_state=0)

In [62]: scale = StandardScaler()
x_train_cls = scale.fit_transform(x_train_cls)
x_test_cls = scale.transform(x_test_cls)

In [63]: # Create the SVR regressor
svr_regressor = SVR()

# Define the parameter grid to search over
param_grid = {
    'kernel': ['linear', 'poly', 'rbf'],
    'C': [0.1, 1.0, 10.0],
    'epsilon': [0.01, 0.1, 1.0]
}

# Initialize the GridSearchCV object
grid_search = GridSearchCV(estimator=svr_regressor, param_grid=param_grid, cv=3, scoring='neg_mean_squared_erro

# Perform hyperparameter tuning
grid_search.fit(x_train_cls, y_train_cls)
```



```
# Get the best hyperparameters and best model
```

```
best_params = grid_search.best_params_  
best_model = grid_search.best_estimator_
```

```
print(best_params)  
print(best_model)
```

```
{'C': 10.0, 'epsilon': 0.01, 'kernel': 'linear'}  
SVR(C=10.0, epsilon=0.01, kernel='linear')
```

```
In [64]: # Retrain the best model on the full training set  
best_model.fit(x_train_cls, y_train_cls)
```

```
# Make predictions
```

```
predict = best_model.predict(x_test_cls)  
predict
```

```
Out[64]: array([ 9917.9676722 , 11887.0837906 , 10797.15539 , 113107.95901743,  
11105.87134894, 11358.47857781, 108119.04899844, 17138.52594084,  
95469.09465333, 105246.21331017, 105696.76454655, 109989.99170628,  
11953.06104054, 13521.46949854, 75658.07895841, 94617.69843402,  
112217.82717838, 10792.21619838, 15729.63002787, 109754.10419582,  
83079.11744924, 83185.0911556 , 11976.39016934, 11523.63679731,  
16564.51848526, 12771.79627539, 18630.65477661, 98986.89314428,  
10461.28316451, 10096.56652937, 76656.07330279, 113798.55389284,  
106930.32320342, 102435.70498292, 10679.65418328, 84322.73316397,  
109168.32573795, 17449.9138181 , 76227.37308198, 11431.70163472,  
15780.99401748, 10335.3065919 , 85195.30879499, 101298.54456272,  
11607.97074128, 17691.90830856, 92799.83121535, 9399.54588875,  
110663.10256888, 17473.49385627, 11038.5364043 , 115504.5940072 ,  
10551.5301333 , 13452.098367 , 86593.08653337, 18134.77680694,  
11807.15954024, 110071.59869592, 17231.87507384, 112685.0568149 ,  
117602.01589185, 109142.04846078, 81733.88350526, 11988.01193886,  
11024.15870141, 117430.94593357, 91685.49235407, 14074.71329426,  
9153.68243399, 10843.85868616, 18229.95773905, 16923.83669714,  
10893.8133782 , 101837.86768606, 114149.49957436, 11938.75358309,  
11286.81412934, 112728.37876841, 85443.21670339, 10769.93678086,  
116496.97042802, 12170.48165932, 101315.57632449, 17320.99021438,  
14438.47660346, 83251.63200559, 118439.84563793, 111639.5796219 ,  
116039.29263152, 11784.54680164, 18516.42487266, 70521.40715611,  
16274.55382346, 94302.29062537, 12263.51526733, 96723.19028686,  
113291.11397473, 118256.79651932, 17784.85353624, 17503.577545 ,  
17355.12817463, 112100.06929742, 18101.60630733, 17060.34452082,  
11824.78862244, 98719.64284522, 78989.70634331, 17726.49779867,  
95585.17487165, 107402.45009556, 17528.74431833, 96293.60602607,  
9462.90029201, 10710.38033673, 83635.10171071, 15057.99899333,  
17265.8942642 , 16638.63840957, 11998.75144826, 101438.02038175,  
114402.32993921, 116970.49467315, 10642.48364112, 17430.81197441,  
107460.48922573, 10948.38396118, 14886.23075014, 10907.43426221,  
10648.25471444, 17212.57336684, 10852.81230325, 16689.07259417,  
104810.91953099, 82766.51176415, 105395.26472613, 17512.43806575,  
102611.51970448, 14761.48983632, 16730.13359181, 99845.38501855,  
9250.15314791, 115494.37329774, 14880.15568891, 116012.3655101 ,  
10444.34764468, 12224.13842988, 11449.59375077, 107915.07094555,  
118004.83474304, 95519.74541679, 12069.75247601, 108222.46311685,  
18012.60811801, 11754.62791908, 10990.24983967, 18217.74976615,  
11814.15117269, 112480.79359174, 103956.28956678, 77318.78004523,  
16827.57267925, 18058.59543237, 11669.09773388, 10574.15972823,  
11737.59014707, 110341.23625129, 10854.49172594, 18604.59563918,  
101067.19611546, 17125.02829466, 16333.63104979, 123821.29417862,  
128270.2745802 , 101937.51922583, 10070.86533102, 17760.74744799,  
79929.56638617, 11333.34430976, 107620.57958148, 118131.10808808,  
102112.98272423, 107303.87787546, 12066.5144572 , 87008.13208081,  
111035.73043162, 17535.49888558, 11587.8093256 , 98532.09925701,  
101394.25603565, 9550.82830987, 80304.08384017, 108173.39041354,  
96327.32606954, 116613.21744383, 104258.72500049, 104319.33330705,  
12396.79424724, 11605.97625309, 17683.89771778, 13525.11569822,  
94986.70223887, 11002.99646164, 77057.39111422, 116827.6350828 ,  
10817.96663474, 14874.30735112, 116430.83990597, 108859.37141995,  
9661.66106189, 10578.02418038, 99053.9782582 , 17490.19706972,  
10385.94465775, 81497.73161221, 84154.92246647, 117050.17120295,  
102092.78000509, 13985.40358042, 16788.66347871, 94821.7388413 ,  
11496.63888912, 98387.07899489, 12206.72978561, 71747.3867925 ,  
102319.63514903, 17175.92762258, 16506.51689138, 19251.26008296,  
17795.89188586, 114223.45481044, 128351.56328114, 11617.83120691,  
17790.37024772, 8902.67904272, 121625.04742789, 17288.20206614,  
11637.92344401, 78137.10991585, 88502.56969245, 99908.22523627,  
112189.52742957, 105394.93324529, 103430.07065446, 110995.65173967,  
121343.45243208, 116940.5224513 , 109045.54704014, 118435.81852136,  
10811.51959278, 14991.34628503, 15673.72667461, 18538.9193149 ,  
10962.00383752, 117648.49332906, 17269.84783713, 91963.09705076,  
10799.64373146, 18039.93663573, 12987.70941359, 17052.90979342,  
103628.46851889, 100126.42075262, 12201.02521181, 16229.99595511,  
83283.6408338 , 117003.7938432 , 17147.90372522, 104749.63000693,
```

75899.21675533, 77679.17153181, 98316.56186994, 103005.25004618,
70712.29826511, 10734.1316548, 133609.93746135, 18171.86479291,
82765.74049627, 10214.42670483, 112415.44885892, 16756.41171914,
85895.08045862, 17119.26373792, 17973.99929085, 11517.34761025,
16941.9216437, 101800.46345416, 94165.58098864, 103077.6517861,
87923.25505395, 11360.06004971, 17082.89202691, 118095.94886599,
10956.52060861, 106750.85176398, 109409.76234258, 15703.51837955,
11502.8513259, 17721.72084176, 100731.65641321, 116461.41038158,
11684.06264778, 111817.36218913, 116925.11609355, 19322.95604986,
83925.14439989, 100829.75531837, 11107.0828446, 14908.14087502,
105815.27973373, 98889.21400501, 116642.65526324, 15809.81657768,
115090.22712719, 11939.77973203, 12250.38542172, 118997.27136112,
17776.43622216, 113306.0531474, 10961.31732977, 100735.5898591,
11701.77116078, 104571.03040846, 17152.73683967, 117465.36531489,
15812.83076291, 17337.2924959, 19378.67644068, 113740.75479511,
18250.42188367, 15865.99952071, 11342.66299211, 10544.06899776,
11685.75794137, 100377.15758983, 110237.93153472, 17505.9413202,
85785.45076869, 17689.0828254, 101214.56043117, 19251.88269788,
12286.18065455, 103350.80803575, 111705.18338822, 17796.67068198,
115114.16342639, 116835.64501665, 17442.65377488, 105855.29100591,
17629.57824891, 85653.57873167, 107119.23148332, 102785.17777092,
12358.44648571, 126689.62465951, 11930.93227339, 10970.55785323,
118966.9996841, 78898.72169138, 99925.53325827, 15044.39467898,
17516.86397785, 96984.26558063, 11707.53054003, 11364.44256954,
110577.54485101, 9367.28092656, 116750.48033555, 11525.18794643,
108829.52175764, 15719.79587006, 94962.29540088, 109779.03428,
17660.58698439, 16181.48764304, 11206.69251022, 17321.34018337,
77931.03301235, 14746.94767928, 12371.52279797, 12831.52831733,
85050.67681463, 18178.38891042, 15795.22685535, 11990.17473769,
105580.30312117, 11843.9248712, 103479.94328881, 76610.11686266,
84407.71450146, 84944.40898387, 111985.05862988, 17367.94875689,
15111.69352452, 105713.08429631, 107690.30433102, 11830.01158752,
110853.96892555, 11434.44517165, 120652.38533969, 10740.74515185,
76311.10434464, 99050.42828471, 84922.59797017, 113489.087362,
74931.07728545, 117509.66818014, 17278.92043515, 12160.18731162,
17697.46052947, 111642.06019339, 102425.31495572, 75322.20552525,
11882.94184889, 98247.222784, 15766.62740811, 96190.85289202,
9252.03994453, 15833.80796349, 11220.06609106, 95773.22308165,
17560.51957013, 11017.3535964, 102767.68187459, 116654.48262392,
17939.2058847, 10687.51164774, 107338.54225436, 17893.87888262,
113258.08776612, 19330.38984547, 11137.84407737, 109133.37199069,
10973.37127648, 95746.64049516, 119149.65913125, 10691.77551825,
112220.56555833, 18109.8334797, 11563.95766062, 106942.91104203,
102234.91434179, 11231.51391683, 103393.04150899, 119061.39000089,
113548.12757593, 9252.34274467, 120697.19849662, 102999.87600251,
11135.38647579, 18382.77541085, 16212.50861807, 117982.60817029,
12165.8019108, 11565.85618777, 11397.88373276, 107055.12461228,
16139.2091597, 75972.85182617, 110621.00177799, 10907.37630717,
10922.13321514, 100961.31536677, 93954.64442973, 110805.08169197,
121639.24455962, 77171.85479868, 14550.64909092, 10958.19696746,
18216.76080413, 99835.37199222, 106142.2405602, 106930.59569401,
10533.26344495, 93478.44902723, 102124.78763646, 17811.27253137,
9772.65221201, 14329.03893646, 127529.03836121, 11944.84552819,
107896.65490247, 116228.63928197, 78536.86178227, 19418.00815752,
17897.9170746, 10409.80760777, 117954.0660805, 84762.53777578,
83342.94965844, 82741.02812776, 95596.70398228, 17085.62842473,
17848.79030242, 87550.37475093, 108172.60499139, 10843.93585073,
11397.17217492, 17202.35864607, 103376.46955262, 92259.44922086,
12770.95519578, 127985.85096786, 17368.93232176, 12266.17780633,
10552.99771531, 109090.31919734, 10073.24412633, 15166.58805163,
101879.87818165, 18386.31770797, 16001.32410712, 97685.58174672,
112996.48377076, 112067.8087262, 105508.04452796, 17290.8886224,
19419.71090543, 17361.64308448, 80813.34235934, 11559.41436193,
14550.33717612, 10274.40847514, 11951.53882319, 80668.32823751,
119644.64890134, 12691.66036027, 11818.22498267, 12188.94112835,
116390.79035664, 119812.92631221, 120301.61850707, 83698.90981871,
103893.08565366, 105378.3428668, 84544.74563578, 125745.74186607,
81666.12056042, 14578.60173389, 10524.40683855, 83599.3165888,
17706.98033617, 10108.95537045, 109796.91935382, 12885.96118152,
17190.48134002, 11746.78693581, 17965.07686797, 17532.42575836,
116541.9029886, 114459.91008795, 17742.97065411, 112362.60627042,
93189.44882054, 11536.87995121, 111612.85957414, 10657.09250905,
101436.61375479, 99142.57478635, 103706.84678545, 97389.3836873,
103544.41406719, 9954.0641983, 11551.80367101])

```
In [65]: print("Mean Absolute Error:", round(metrics.mean_absolute_error(y_test_cls, predict), 4))
print("Mean Squared Error:", round(metrics.mean_squared_error(y_test_cls, predict), 4))
print("Root Mean Squared Error:", round(np.sqrt(metrics.mean_squared_error(y_test_cls, predict)), 4))
print("(R^2) Score:", round(metrics.r2_score(y_test_cls, predict), 4))
print(f'Train Score : {model.score(x_train_cls, y_train_cls) * 100:.2f}% and Test Score : {model.score(x_test_cls, y_test_cls) * 100:.2f}%')
errors = abs(predict - y_test_cls)
mape = 100 * (errors / y_test_cls)
accuracy = 100 - np.mean(mape)
```

```
print('Accuracy:', round(accuracy, 2), '%.')
```

Mean Absolute Error: 584.4827

Mean Squared Error: 1718588.7716

Root Mean Squared Error: 1310.9496

(R²) Score: 0.9992

Train Score : 100.00% and Test Score : 99.87% using SVR Clustering Regressor.

Accuracy: 99.02 %.

```
In [66]: predictions = pd.DataFrame({"Predictions": predict})
predictions['Date'] = pd.date_range(start=nse_data.index[-1], periods=len(predict), freq="D")
predictions.set_index("Date", inplace=True)
predictions.to_csv("Predicted-price-data_cls_NSE.csv")
#collects future days from predicted values
oneyear_df = pd.DataFrame(predictions[:252])
oneyear_df.to_csv("one-year-predictions_cls_NSE.csv")
onemonth_df = pd.DataFrame(predictions[:21])
onemonth_df.to_csv("one-month-predictions_cls_NSE.csv")
fivedays_df = pd.DataFrame(predictions[:5])
fivedays_df.to_csv("five-days-predictions_cls_NSE.csv")
```

```
In [67]: oneyear_df_pred = pd.read_csv("one-year-predictions_cls_NSE.csv")
oneyear_df_pred.set_index("Date", inplace=True)
buy_price = min(oneyear_df_pred["Predictions"])
sell_price = max(oneyear_df_pred["Predictions"])
oneyear_buy = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == buy_price]
oneyear_sell = oneyear_df_pred.loc[oneyear_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(oneyear_buy)
print("Sell price and date")
print(oneyear_sell)
oneyear_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 year Clustering", color="blue")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date

Predictions

Date

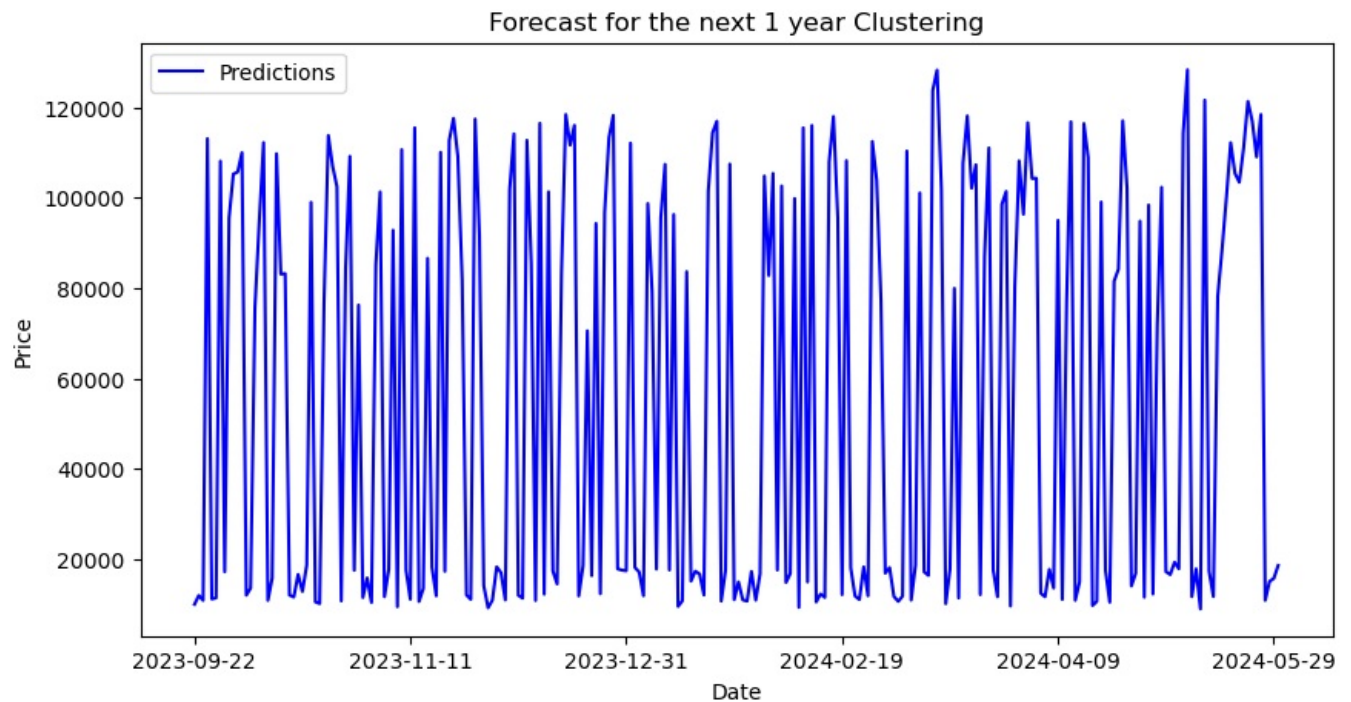
2024-05-12 8902.679043

Sell price and date

Predictions

Date

2024-05-09 128351.563281



```
In [68]: onemonth_df_pred = pd.read_csv("one-month-predictions_cls_NSE.csv")
onemonth_df_pred.set_index("Date", inplace=True)
buy_price = min(onemonth_df_pred["Predictions"])
sell_price = max(onemonth_df_pred["Predictions"])
onemonth_buy = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == buy_price]
onemonth_sell = onemonth_df_pred.loc[onemonth_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(onemonth_buy)
print("Sell price and date")
print(onemonth_sell)
```

```
onemonth_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 1 month Clustering", color="b")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date

Predictions

Date

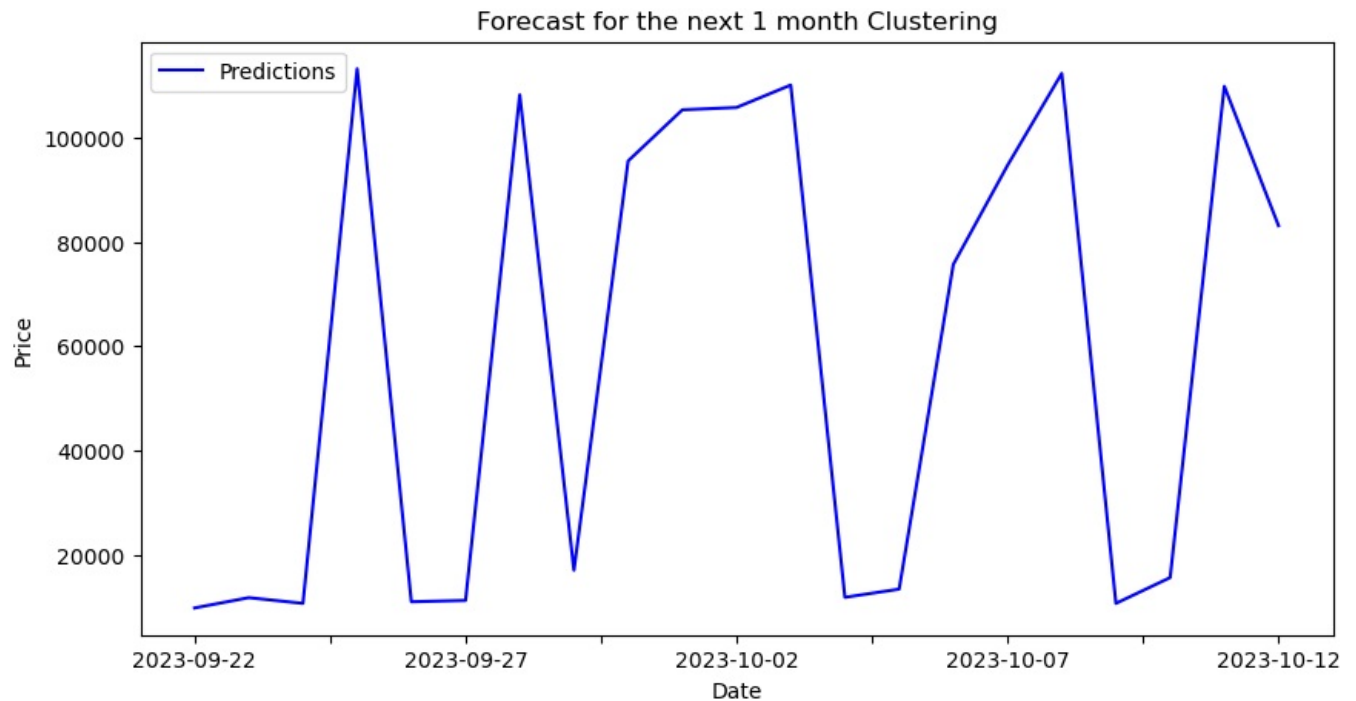
2023-09-22 9917.967672

Sell price and date

Predictions

Date

2023-09-25 113107.959017



```
In [69]: fivedays_df_pred = pd.read_csv("five-days-predictions_cls_NSE.csv")
fivedays_df_pred.set_index("Date", inplace=True)
buy_price = min(fivedays_df_pred["Predictions"])
sell_price = max(fivedays_df_pred["Predictions"])
fivedays_buy = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == buy_price]
fivedays_sell = fivedays_df_pred.loc[fivedays_df_pred["Predictions"] == sell_price]
print("Buy price and date")
print(fivedays_buy)
print("Sell price and date")
print(fivedays_sell)
fivedays_df_pred["Predictions"].plot(figsize=(10, 5), title="Forecast for the next 5 days Clustering", color="b")
plt.xlabel("Date")
plt.ylabel("Price")
plt.legend()
plt.show()
```

Buy price and date

Predictions

Date

2023-09-22 9917.967672

Sell price and date

Predictions

Date

2023-09-25 113107.959017

